

AI

AI VIET NAM
@aivietnam.edu.vn

SOFTMAX REGRESSION

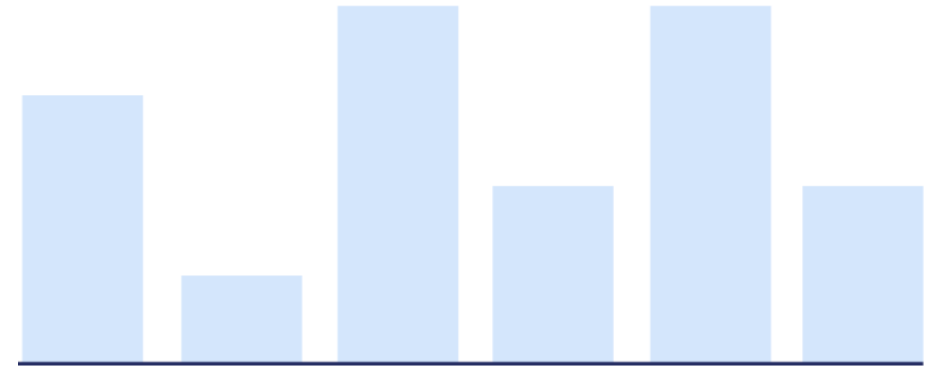
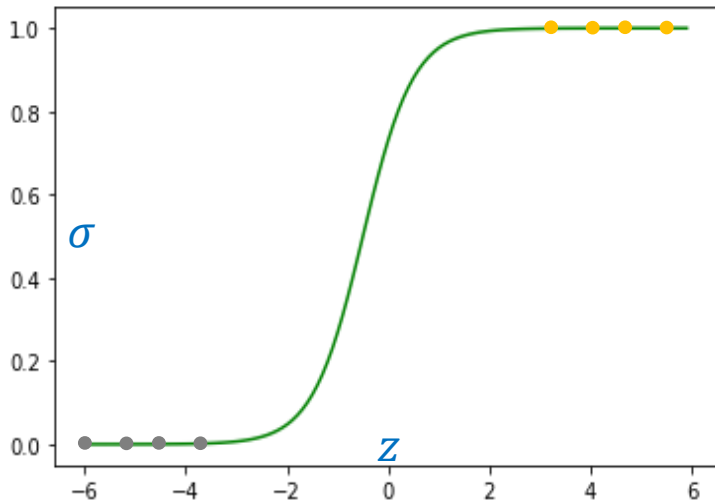
TA. Nguyen Dang Nha
TA. Nguyen Quoc Thai

Outline

Logistic Regression (Review)

1. Logistic Regression
2. Sigmoid Function
3. Gradient Descent

Section 1



Softmax Regression

1. Softmax Regression
2. Softmax Function
3. One Sample
4. N Sample
5. Practice Code (numpy)

Section 2

[Quiz&Feedbacks](#)

Logistic Regression



Logistic Regression

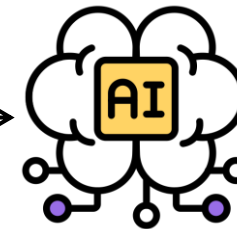
❖ Regression

Dataset

Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1



Study Time



Prediction



Result

Learning

Logistic Regression

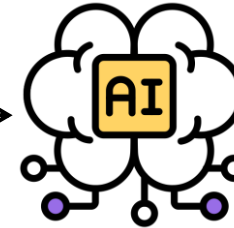
❖ Regression

Dataset

Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1

Input

Study Time



Prediction

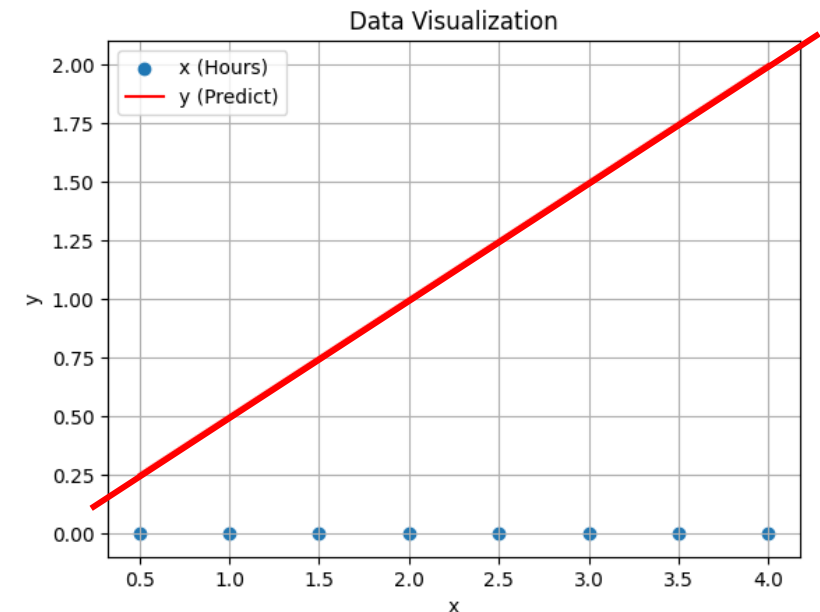
Output

Result

$$\theta = 0.5$$

$$f(x) = h = \theta^T x \quad \text{Liner Regression}$$

$$\begin{aligned} 0.5 * \theta &= 0.25 \\ 1.0 * \theta &= 0.5 \\ 1.5 * \theta &= 0.75 \\ 2.0 * \theta &= 1.0 \\ 2.5 * \theta &= 1.25 \\ 3.0 * \theta &= 1.5 \\ 3.5 * \theta &= 1.75 \\ 4.0 * \theta &= 2.0 \end{aligned}$$

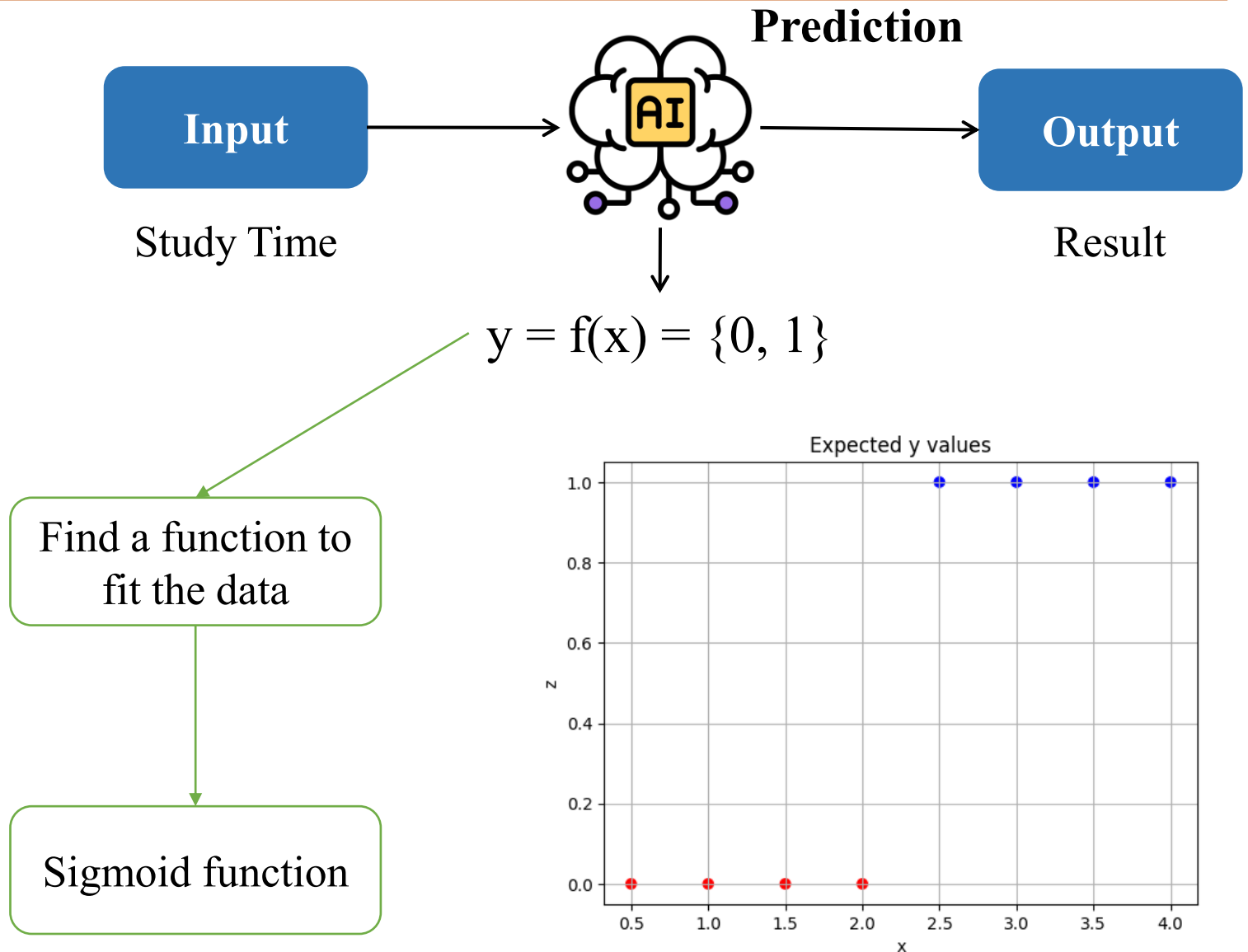


Logistic Regression

❖ Binary Classification

Dataset

Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1



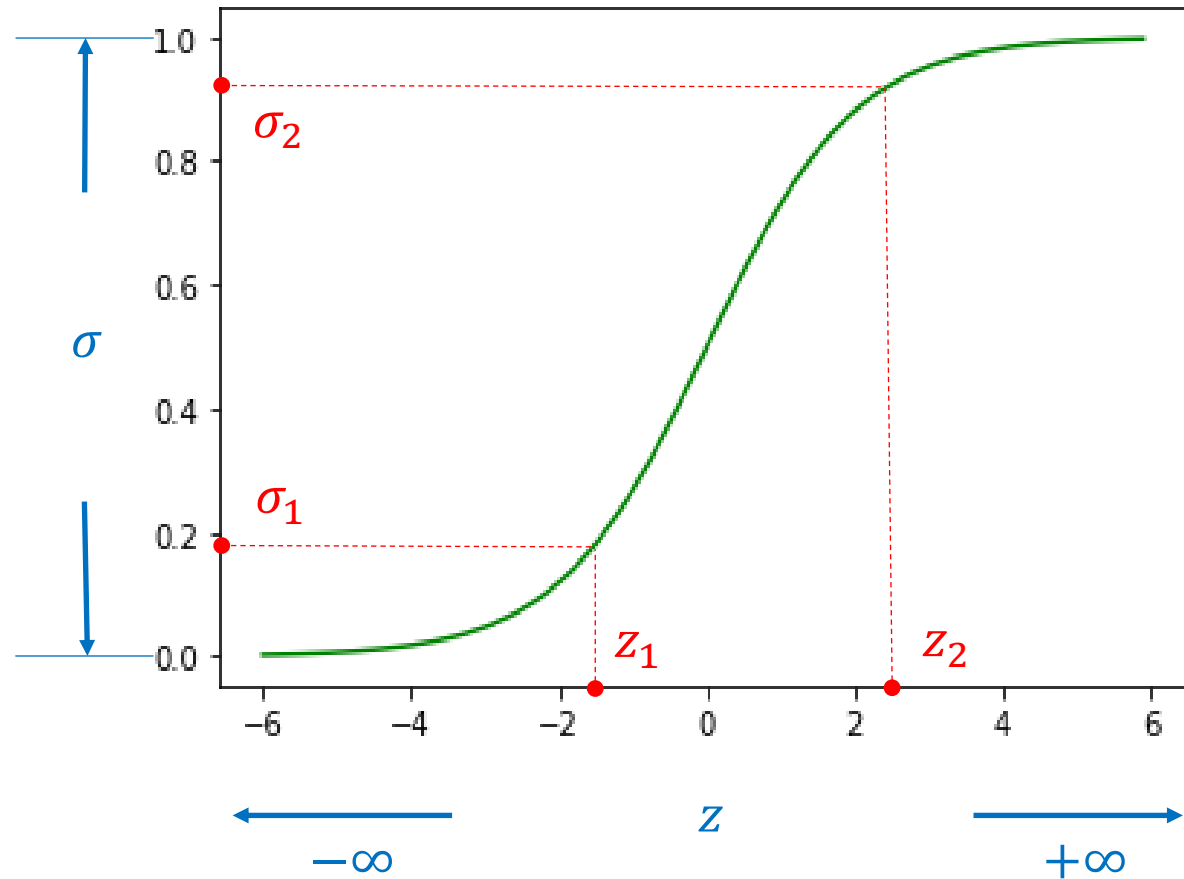
Logistic Regression

❖ Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z \in (-\infty, +\infty)$$

$$\sigma(z) \in (0, 1)$$

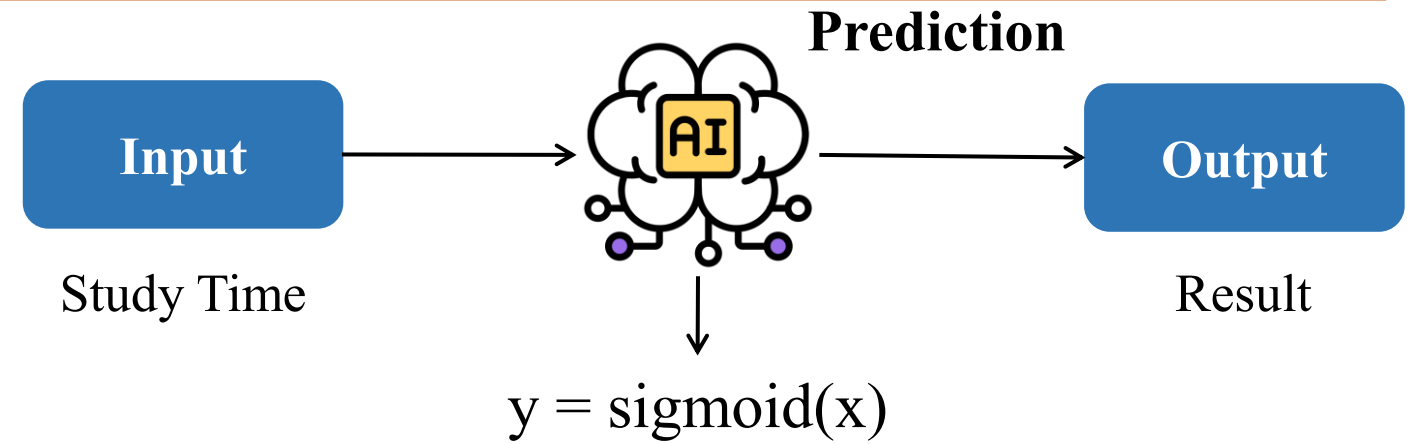


Logistic Regression

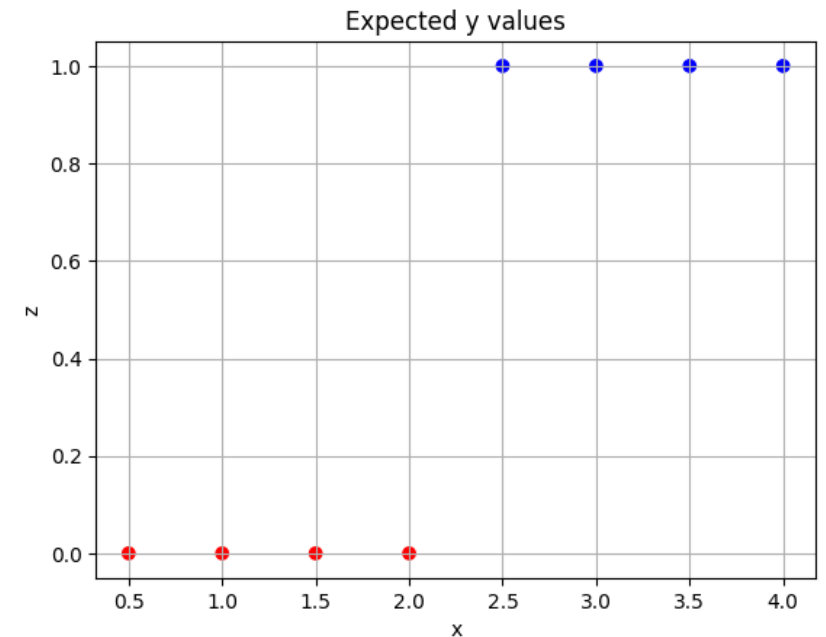
❖ Binary Classification

Dataset

Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1



Learnable
Parameters ?

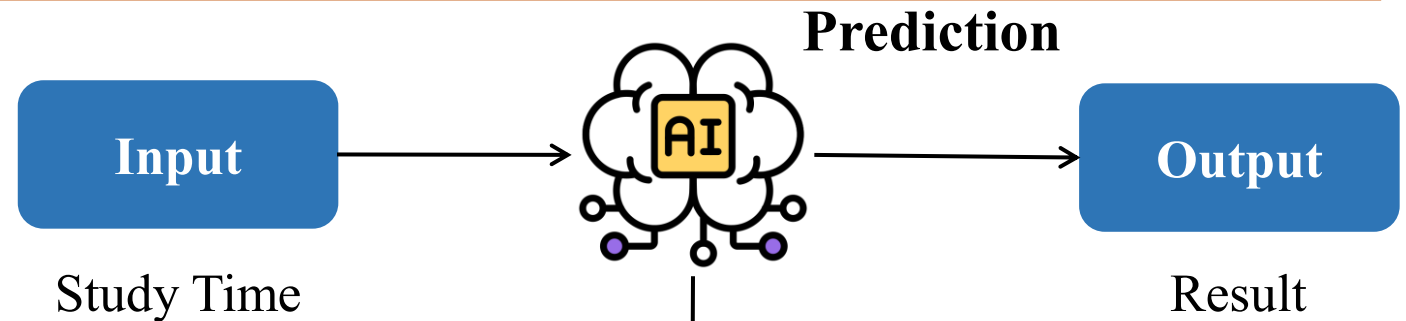


Logistic Regression

❖ Binary Classification

Dataset

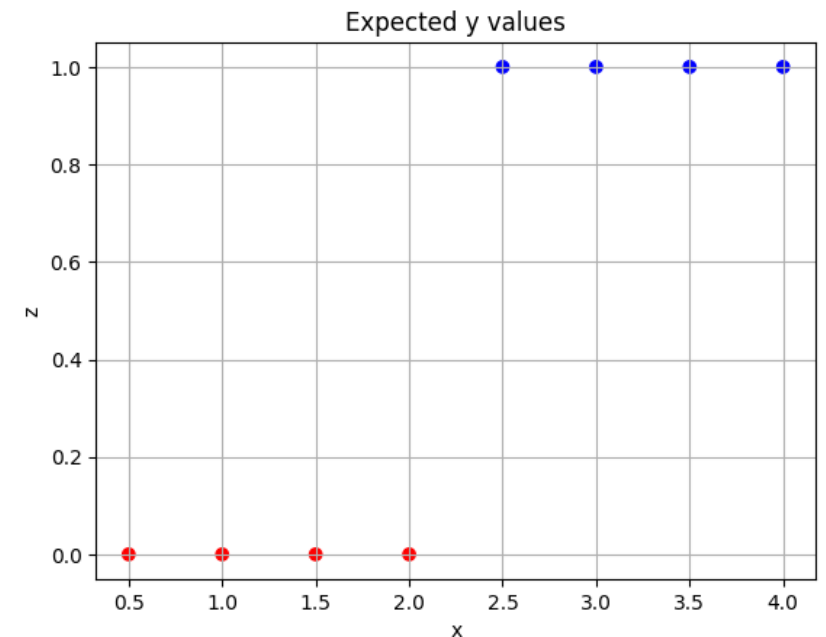
Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1



$$y = \text{sigmoid}(h)$$
$$h = \theta^T x$$

$$\text{Loss} = y - \hat{y}$$

$$\theta = \theta - \nabla \text{Loss}$$



Logistic Regression

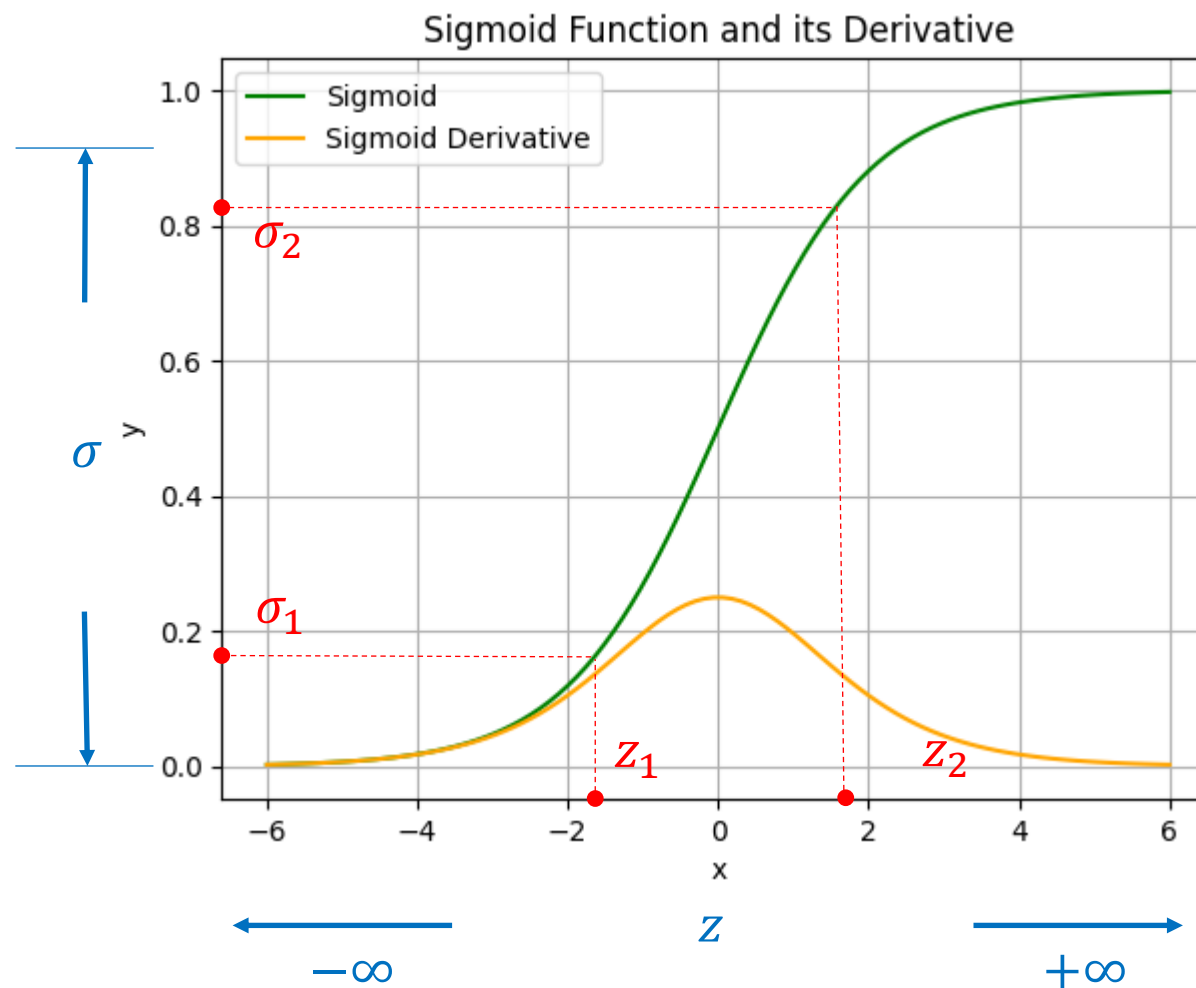
❖ Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$\sigma'(z) = \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



Logistic Regression

❖ Logistic Regression using Gradient Descent

1) Pick a sample (x, y) from training

2) Compute output $\hat{y}$

$$z = \theta^T x$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3) Compute loss

$$L(\theta) = (-y \log \hat{y} - (1 - y) \log(1 - \hat{y}))$$

4) Compute derivative

$$\nabla_{\theta} L = x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate

$$\theta = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$x = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$y = [0]$$

$$\eta = 0.1$$

Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1

Logistic Regression

❖ Logistic Regression using Gradient Descent

1) Pick a sample (x, y) from training

2) Compute output $\hat{y}$

$$z = \theta^T x$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3) Compute loss

$$L(\theta) = (-y \log \hat{y} - (1 - y) \log(1 - \hat{y}))$$

4) Compute derivative

$$\nabla_{\theta} L = x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

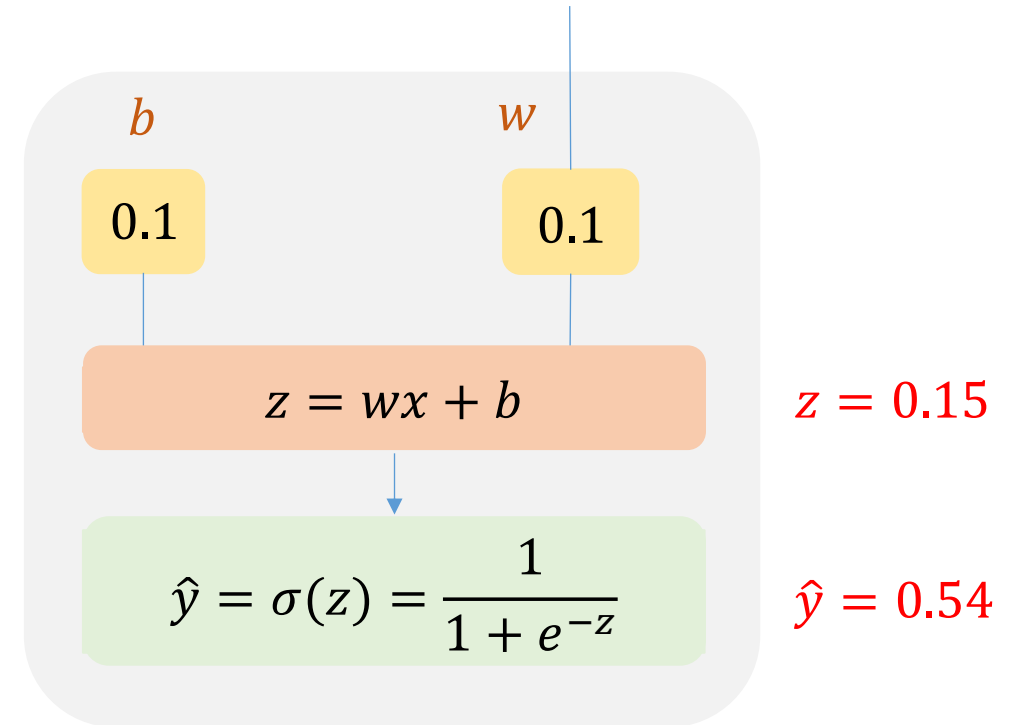
η is learning rate

$$\eta = 0.1$$

$$\theta = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$x = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$y = [0]$$



Logistic Regression

❖ Logistic Regression using Gradient Descent

1) Pick a sample (x, y) from training

2) Compute output $\hat{y}$

$$z = \theta^T x$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3) Compute loss

$$L(\theta) = (-y \log \hat{y} - (1 - y) \log(1 - \hat{y}))$$

4) Compute derivative

$$\nabla_{\theta} L = x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

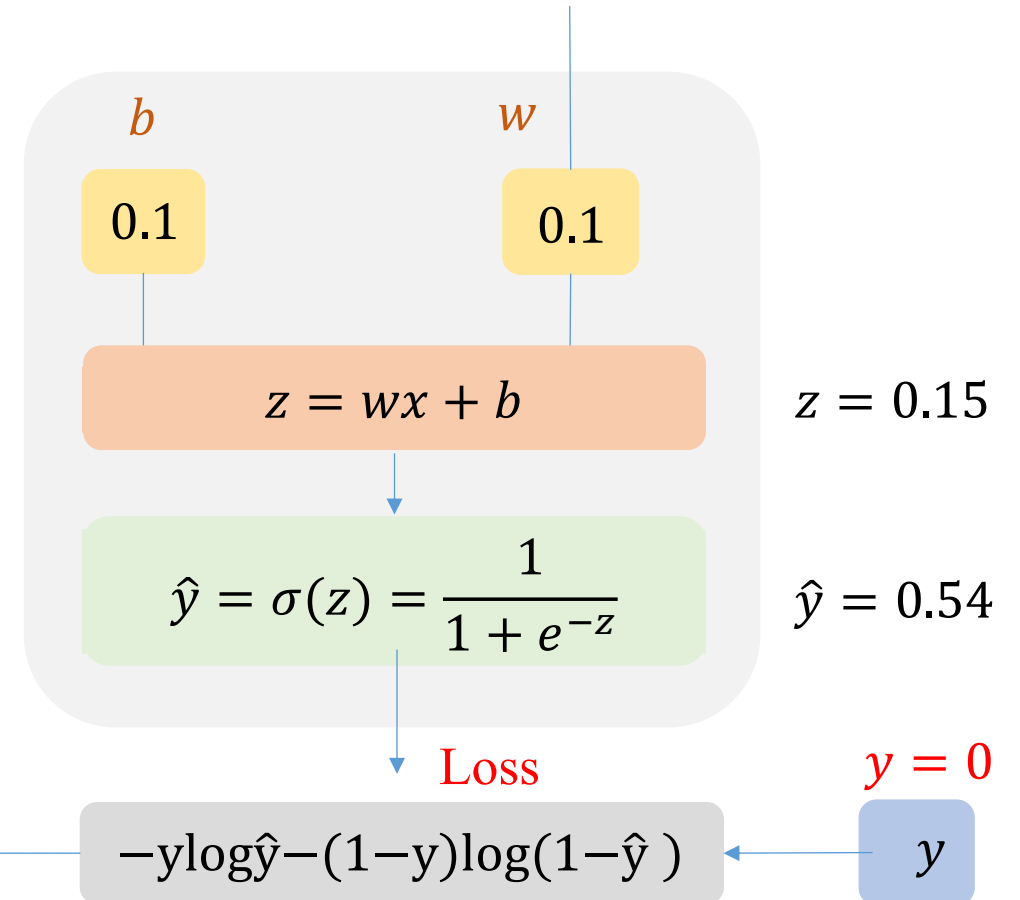
η is learning rate

$$\eta = 0.1$$

$$\theta = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$x = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$y = [0]$$



Logistic Regression

❖ Logistic Regression using Gradient Descent

1) Pick a sample (x, y) from training

2) Compute output $\hat{y}$

$$z = \theta^T x$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3) Compute loss

$$L(\theta) = (-y \log \hat{y} - (1-y) \log(1-\hat{y}))$$

4) Compute derivative

$$\nabla_{\theta} L = x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate

$$\eta = 0.1$$

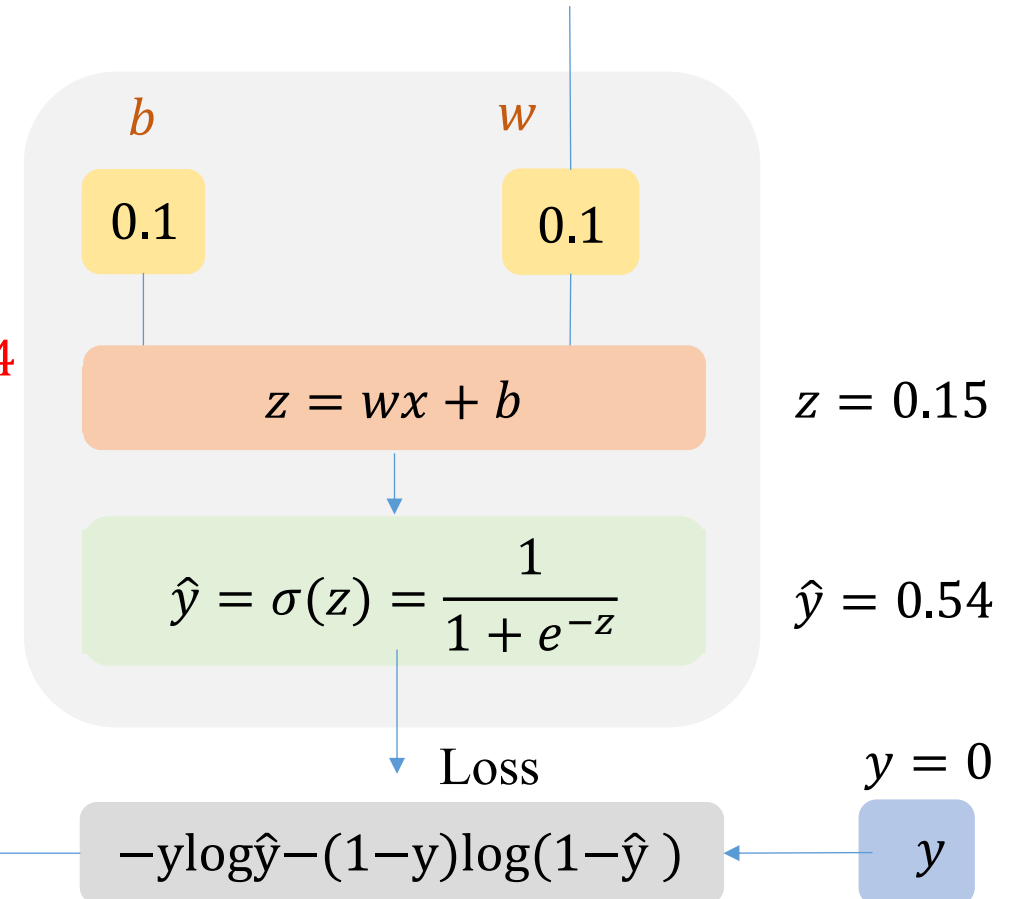
$$\theta = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$x = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$y = [0]$$

$$\begin{aligned} \nabla_{\theta} L &= x(\hat{y} - y) \\ &= \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} * 0.54 \\ &= \begin{bmatrix} 0.54 \\ 0.27 \end{bmatrix} \end{aligned}$$

$$L = 0.771$$



Logistic Regression

❖ Logistic Regression using Gradient Descent

1) Pick a sample (x, y) from training

2) Compute output $\hat{y}$

$$z = \theta^T x$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3) Compute loss

$$L(\theta) = (-y \log \hat{y} - (1-y) \log(1-\hat{y}))$$

4) Compute derivative

$$\nabla_{\theta} L = x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate

$$\eta = 0.1$$

$$\theta = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$x = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$y = [0]$$

$$\theta = \theta - \eta \nabla_{\theta} L$$

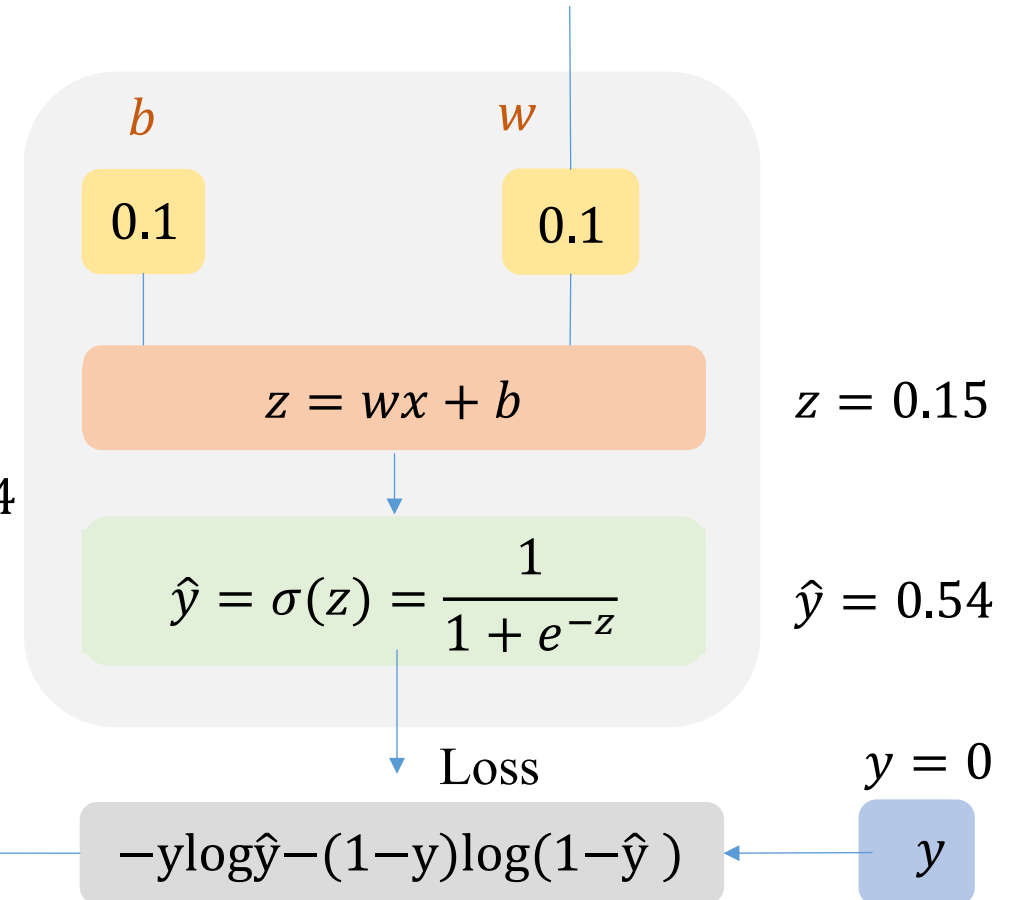
$$= \begin{bmatrix} 0.046 \\ 0.073 \end{bmatrix}$$

$$\nabla_{\theta} L = x(\hat{y} - y)$$

$$= \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} * 0.54$$

$$= \begin{bmatrix} 0.54 \\ 0.27 \end{bmatrix}$$

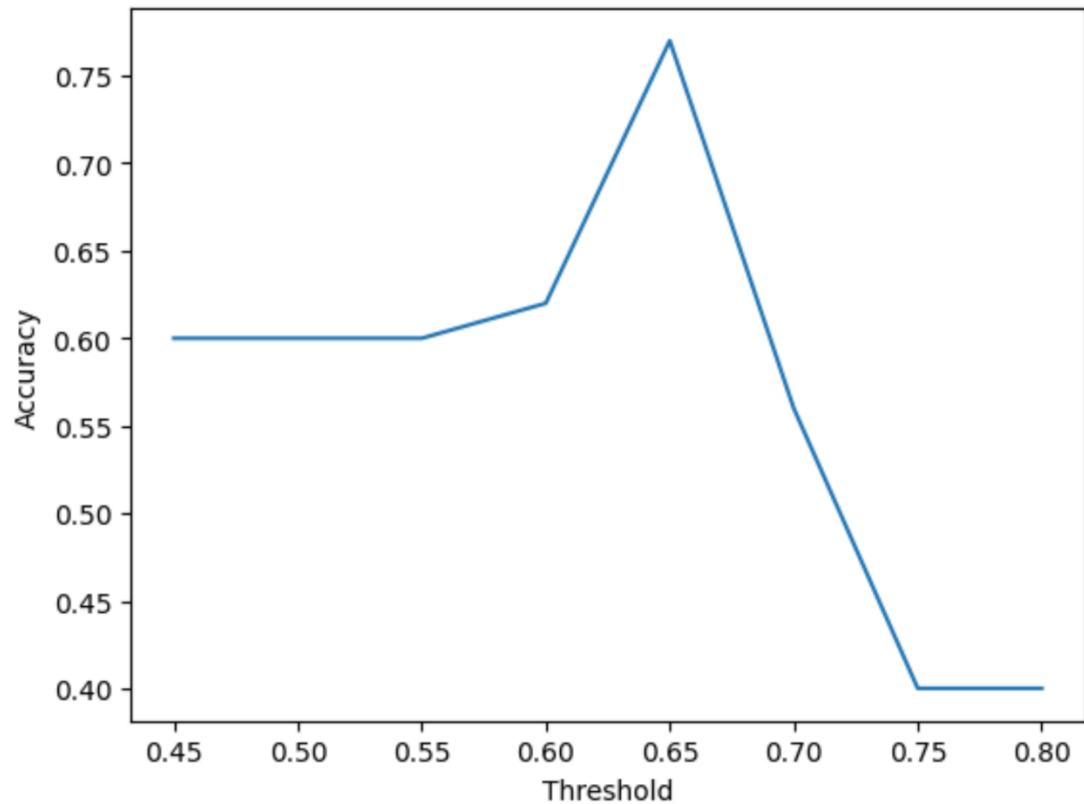
$$L = 0.771$$



Softmax Regression

Softmax Regression

❖ Problem

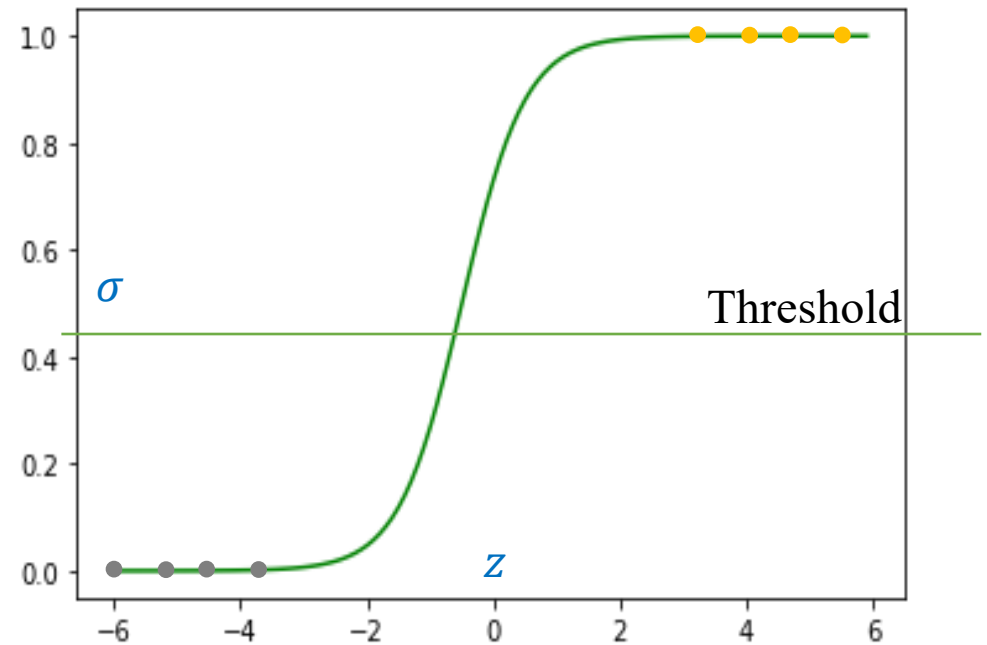


Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z \in (-\infty, +\infty)$$

$$\sigma(z) \in (0, 1)$$



Softmax Regression

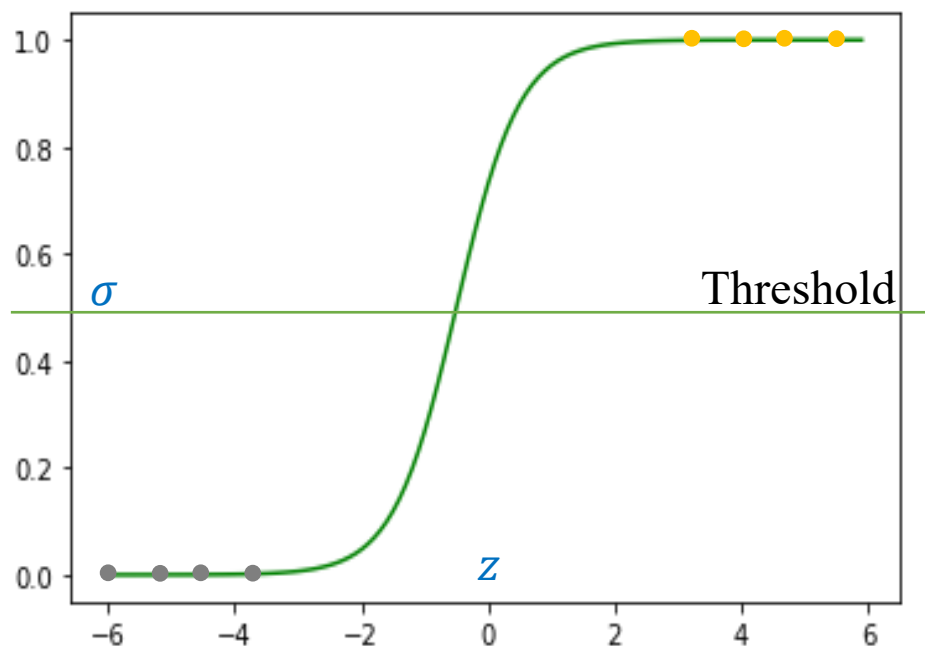
❖ Problem

Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z \in (-\infty, +\infty)$$

$$\sigma(z) \in (0, 1)$$



Hours	Pass
0.5	0
1.0	0
1.5	1
2.0	1

Classes: {0, 1}
Binary Classification

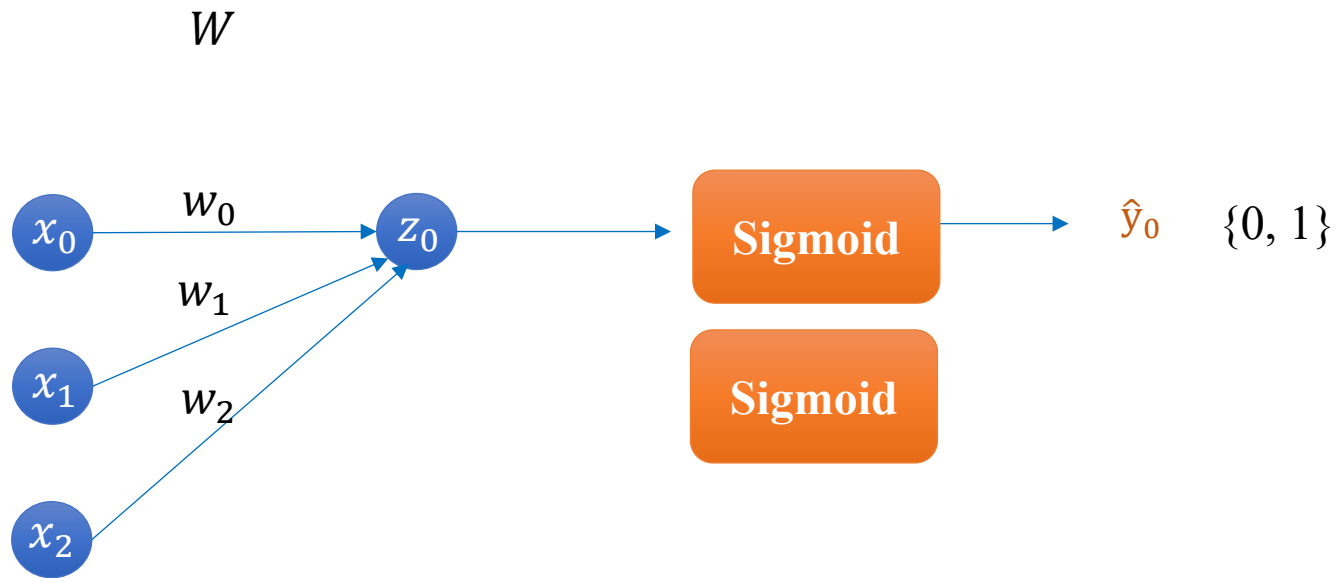
Hours	Score
0.5	0
1.0	0
1.5	1
2.0	1
2.5	2
3.0	2
3.5	3
4.0	3

Classes: {0, 1, 2, 3}
Multi-class Classification

Softmax Regression

❖ Problem

multinomial logistic regression



Hours	Score
0.5	0
1.0	0
1.5	1
2.0	1
2.5	2
3.0	2
3.5	3
4.0	3

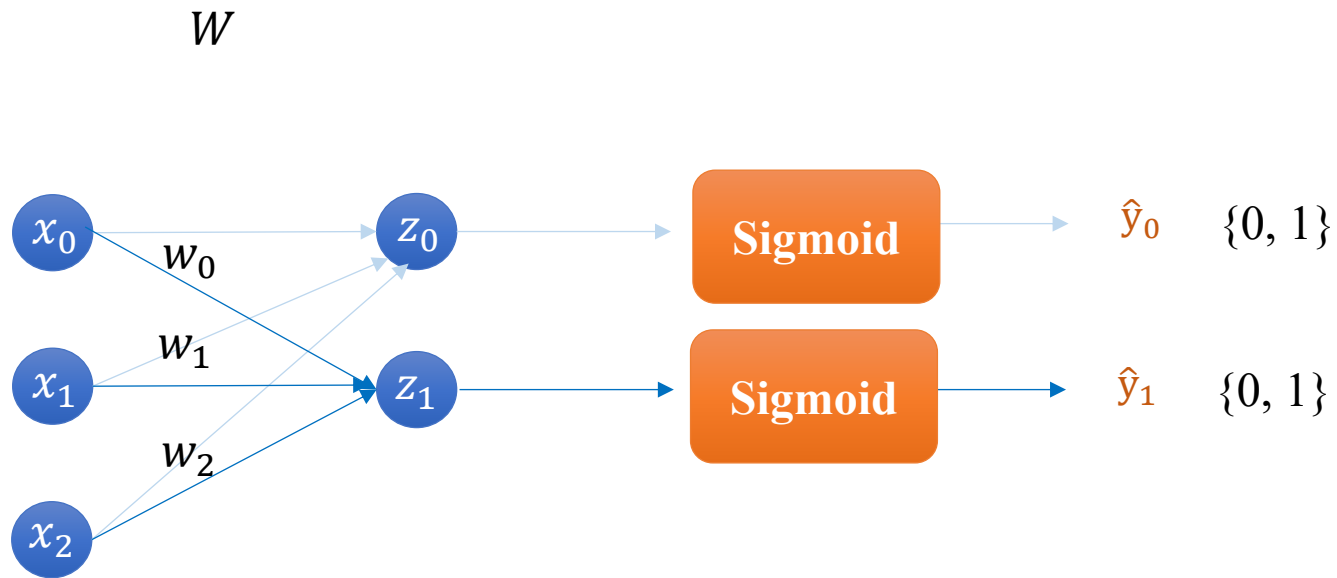
Classes: $\{0, 1, 2, 3\}$
Multi-class Classification

one vs rest

Softmax Regression

❖ Problem

multinomial logistic regression



Hours	Score
0.5	0
1.0	0
1.5	1
2.0	1
2.5	2
3.0	2
3.5	3
4.0	3

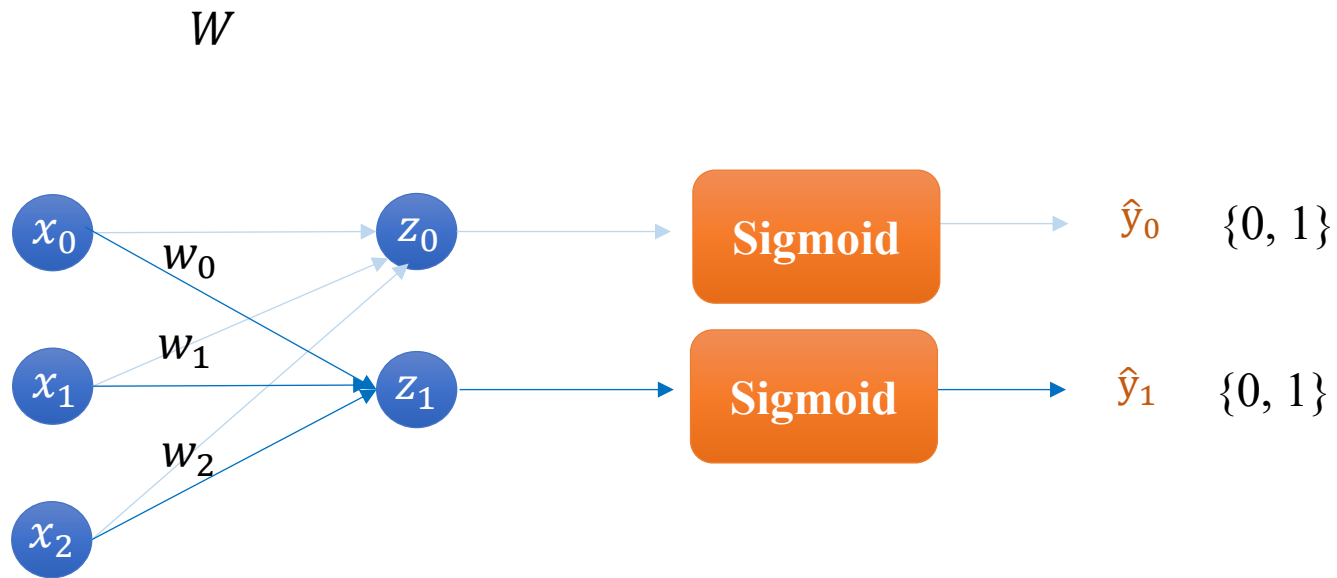
one vs rest

Classes: $\{0, 1, 2, 3\}$
Multi-class Classification

Softmax Regression

❖ Problem

multinomial logistic regression



Problem?

one vs rest

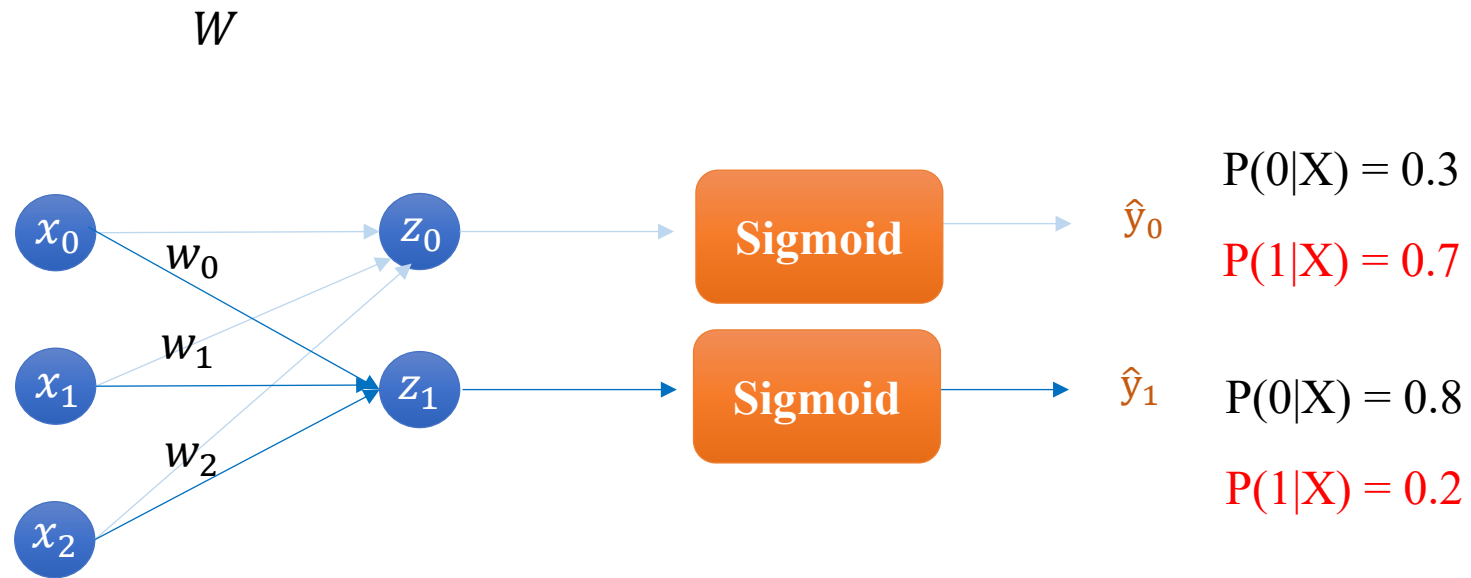
Hours	Score
0.5	0
1.0	0
1.5	1
2.0	1
2.5	2
3.0	2
3.5	3
4.0	3

Classes: $\{0, 1, 2, 3\}$
Multi-class Classification

Softmax Regression

❖ Problem

multinomial logistic regression



Problem?

one vs rest

Hours	Score
0.5	0
1.0	0
1.5	1
2.0	1
2.5	2
3.0	2
3.5	3
4.0	3

Classes: $\{0, 1, 2, 3\}$
Multi-class Classification

Softmax Regression

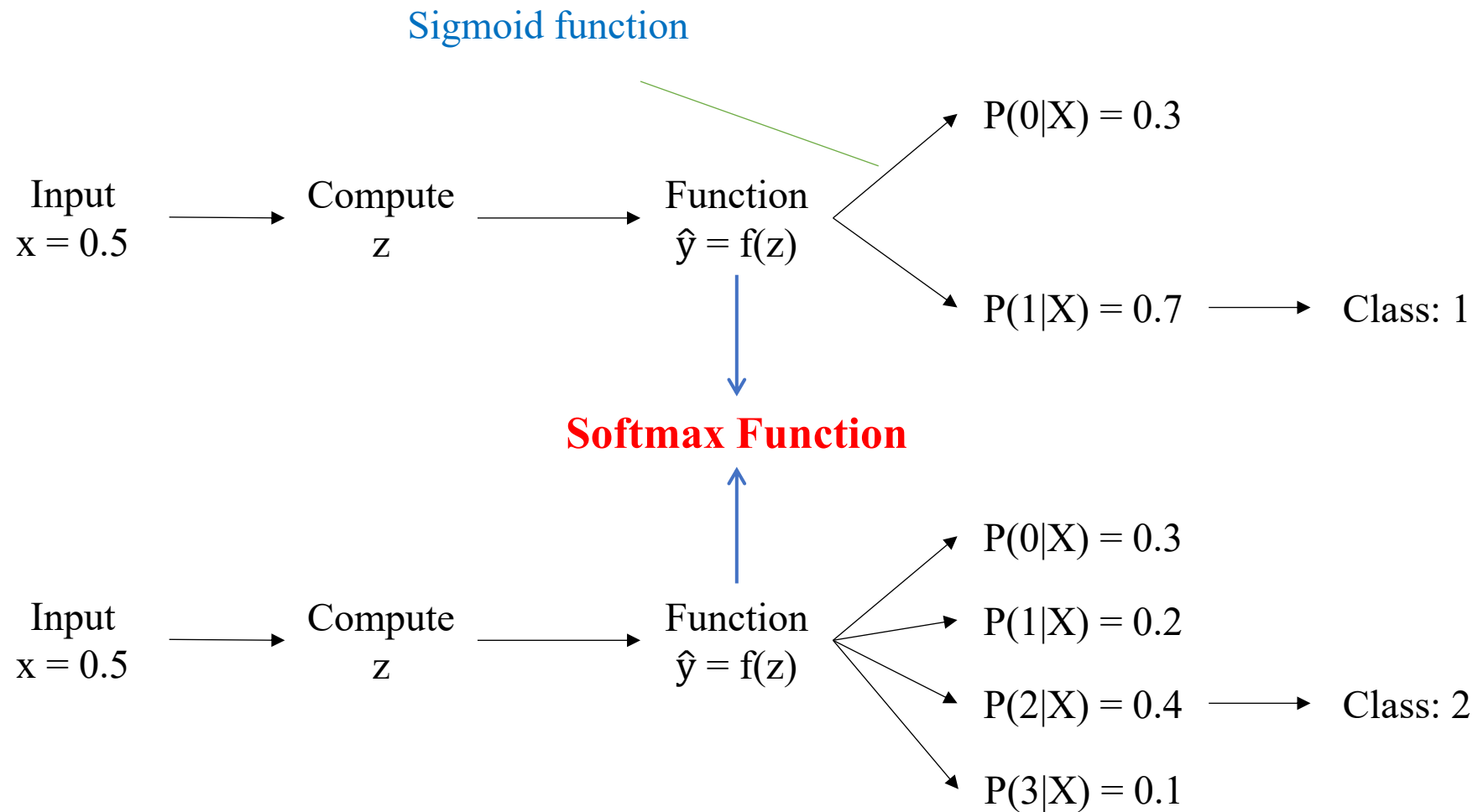
❖ Problem

Classes: {0, 1}
Binary Classification

Hours	Pass
0.5	0
2.0	1

Classes: {0, 1, 2, 3}
Multi-class Classification

Hours	Score
0.5	0
1.5	1
3.0	2
4.0	3



multinomial logistic regression

Softmax Regression

❖ Softmax Function

$$P_i = f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$0 \leq f(z_i) \leq 1$$

$$\sum_i f(z_i) = 1$$

Input

$$z_1 = 1.0$$

$$z_2 = 3.0$$



Softmax



Probability

$$f(z_1) = 0.12$$

$$f(z_2) = 0.88$$

Input

$$z_1 = 1.0$$

$$z_2 = 2.0$$

$$z_3 = 3.0$$



Softmax



Probability

$$f(z_1) = 0.09$$

$$f(z_2) = 0.24$$

$$f(z_3) = 0.67$$

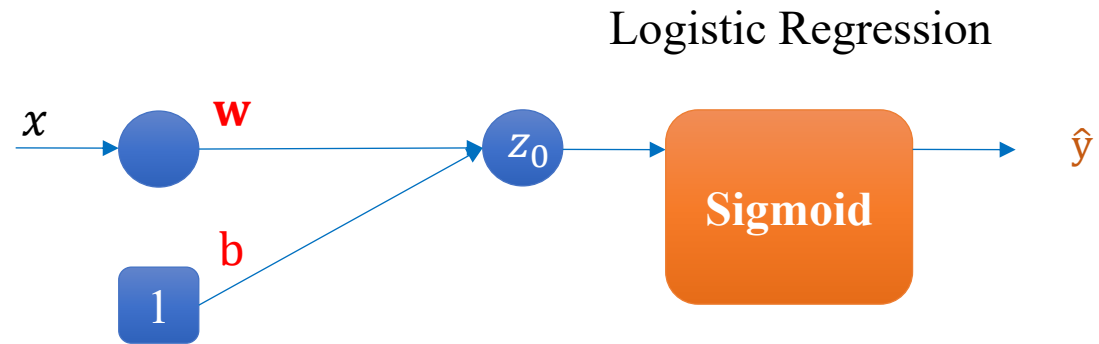
Softmax Regression

❖ Loss Function

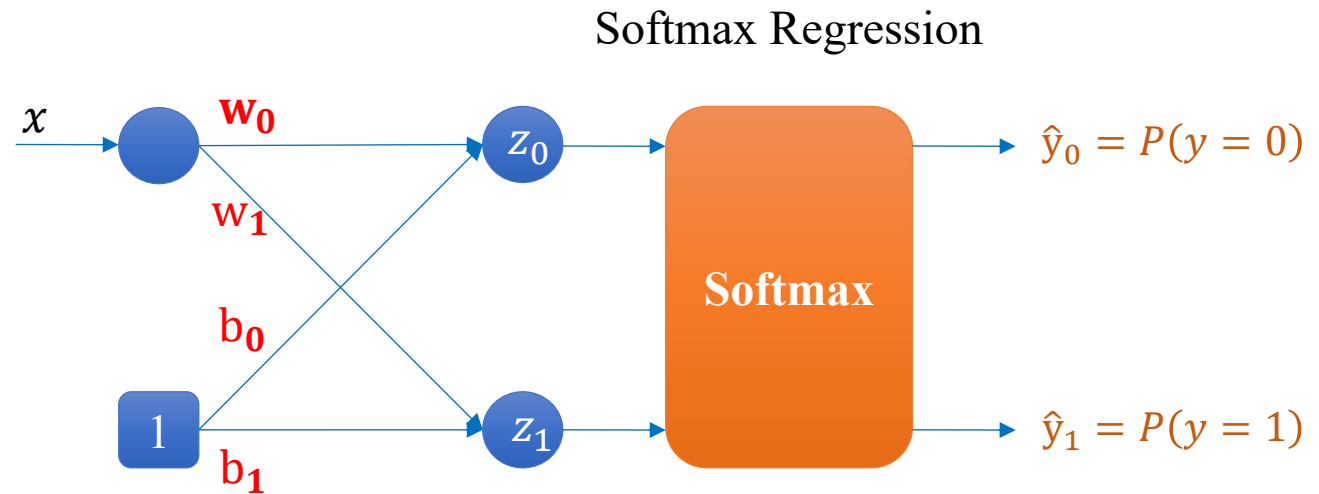
Classes: {0, 1}
Binary Classification

Hours	Pass
0.5	0
2.0	1

#feature: 1
#class: 2



$$P_i = f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



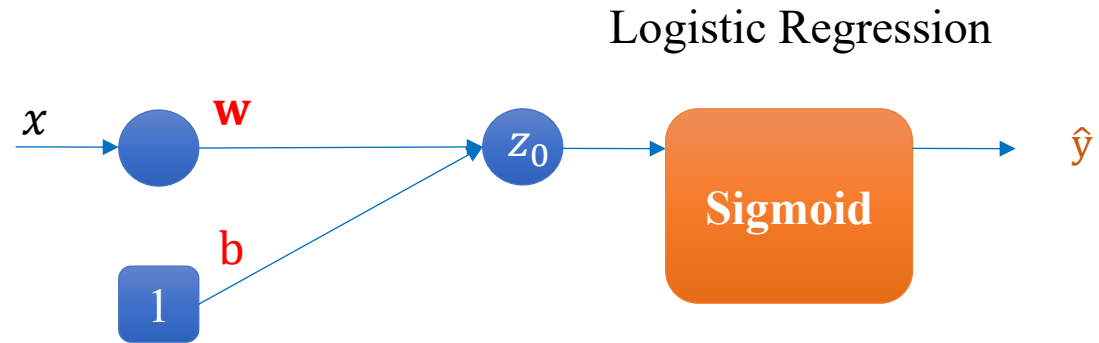
Softmax Regression

❖ Loss Function

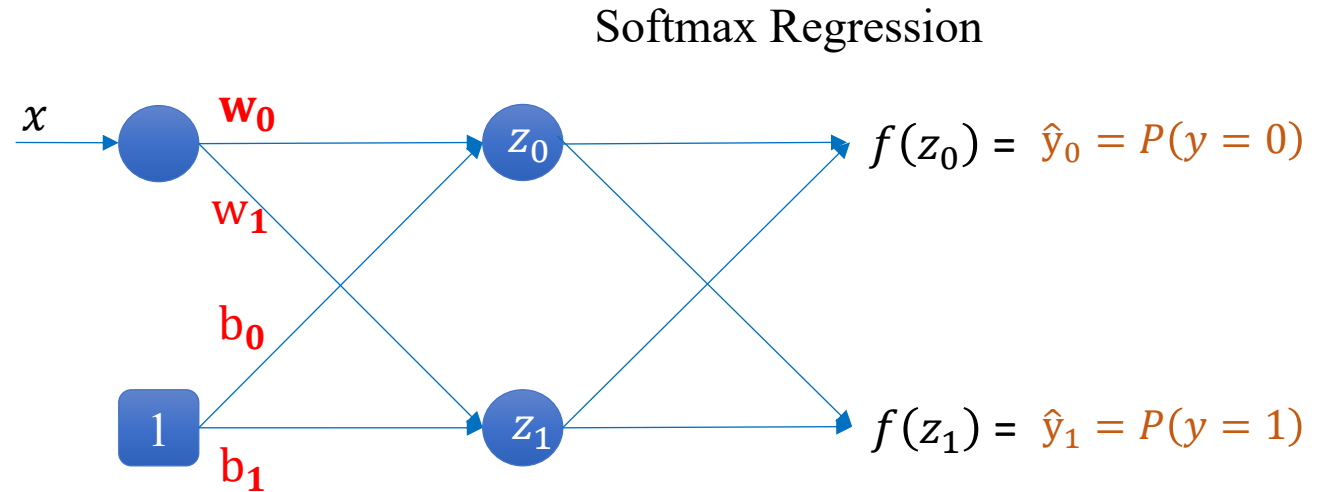
Classes: {0, 1}
Binary Classification

Hours	Pass
0.5	0
2.0	1

#feature: 1
#class: 2



$$P_i = f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



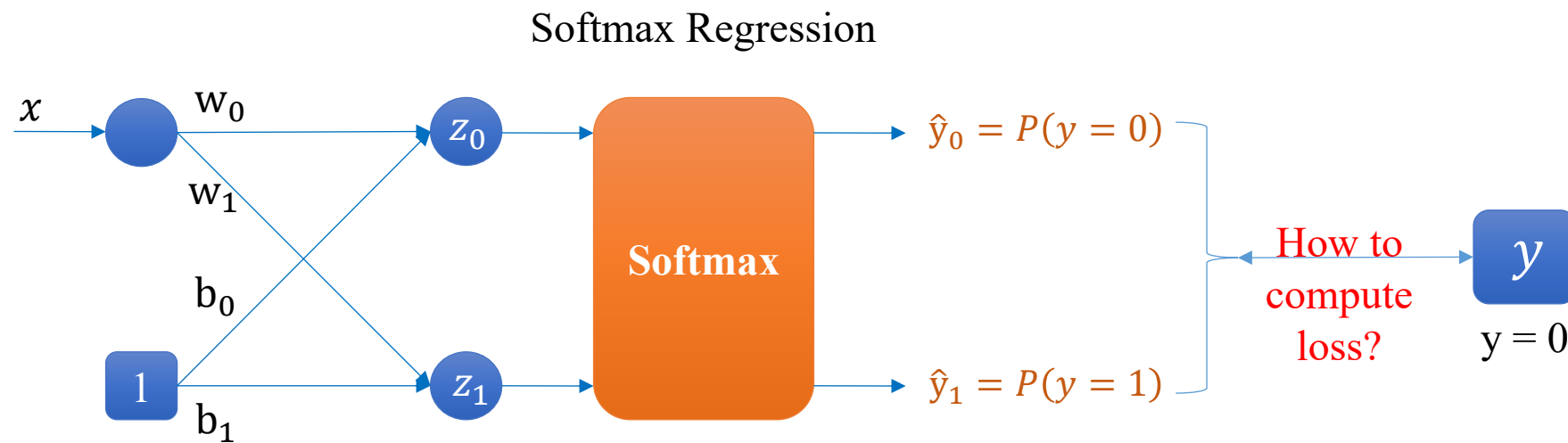
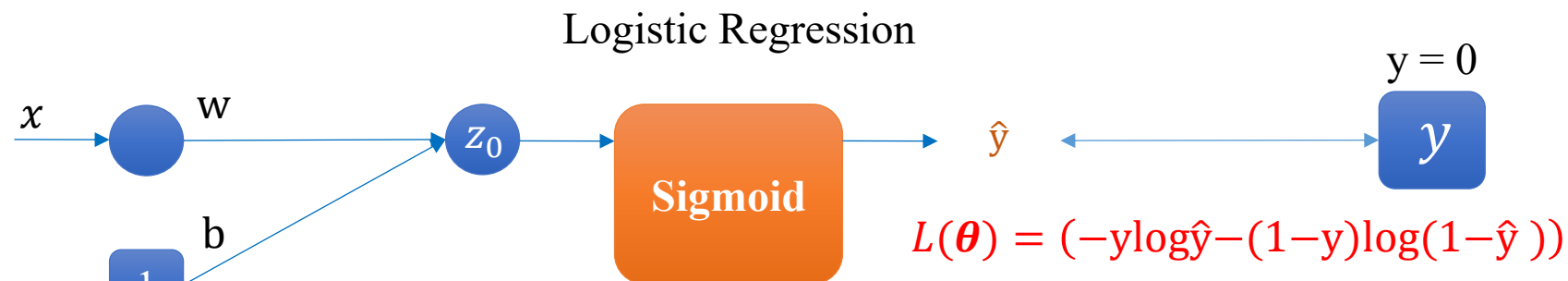
Softmax Regression

❖ Loss Function

Classes: {0, 1}
Binary Classification

Hours	Pass
0.5	0
2.0	1

#feature: 1
#class: 2



Softmax Regression

❖ One-hot Encoding

$$\mathbf{y} = \begin{bmatrix} y_0 \\ \vdots \\ y_C \end{bmatrix}$$

$$y_i \in \{0,1\}$$

$$\sum_i y_i = 1$$

$$C = \text{\#classes}$$

Classes: {0, 1}
Binary Classification

Hours	Pass
0.5	0
2.0	1

#feature: 1
#class: 2

$$y = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$y = 1 \rightarrow \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Classes: {0, 1, 2}
Multi-class Classification

Hours	Score
0.5	0
1.5	1
3.0	2

#feature: 1
#class: 3

$$y = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$y = 1 \rightarrow \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$y = 2 \rightarrow \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

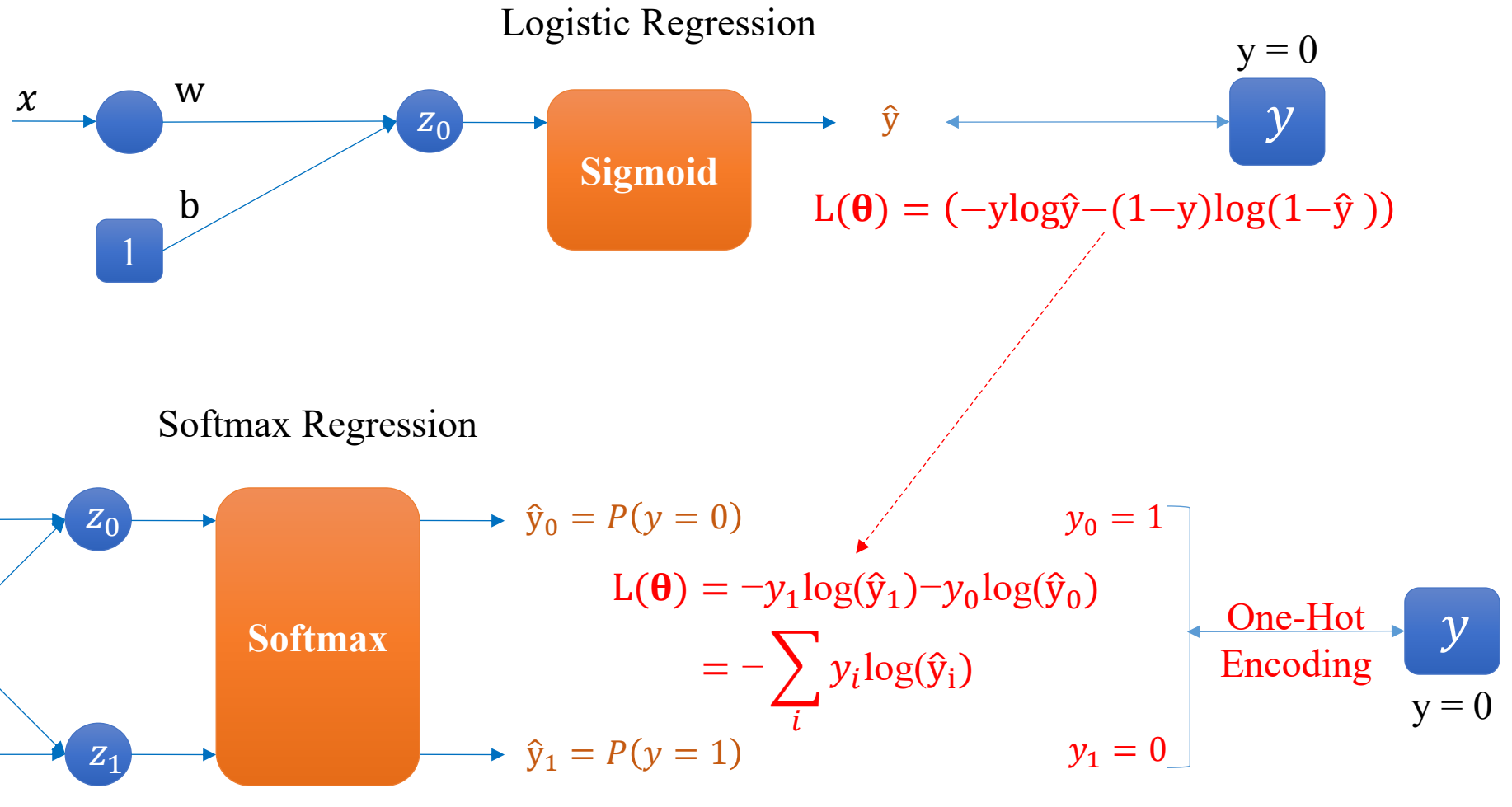
Softmax Regression

❖ Loss Function

Classes: {0, 1}
Binary Classification

Hours	Pass
0.5	0
2.0	1

#feature: 1
#class: 2

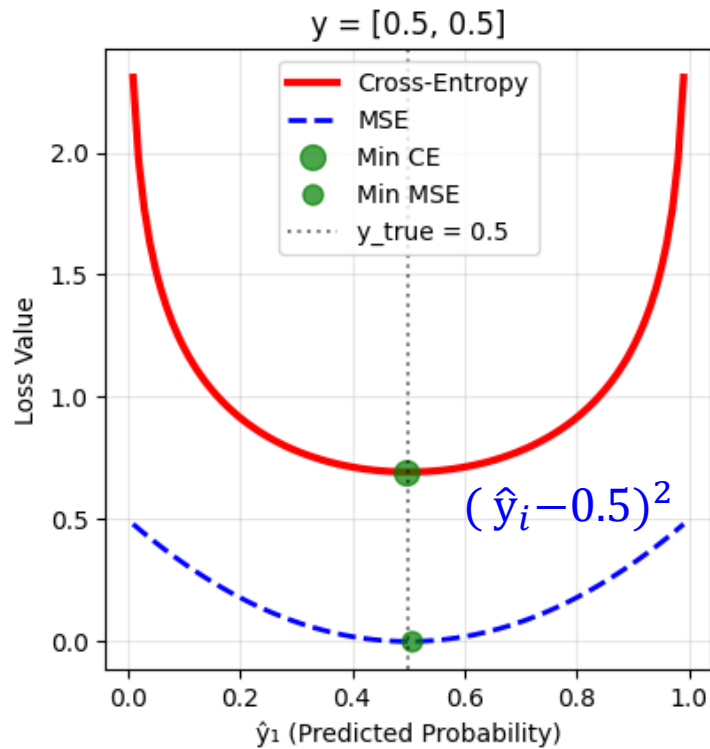


Softmax Regression

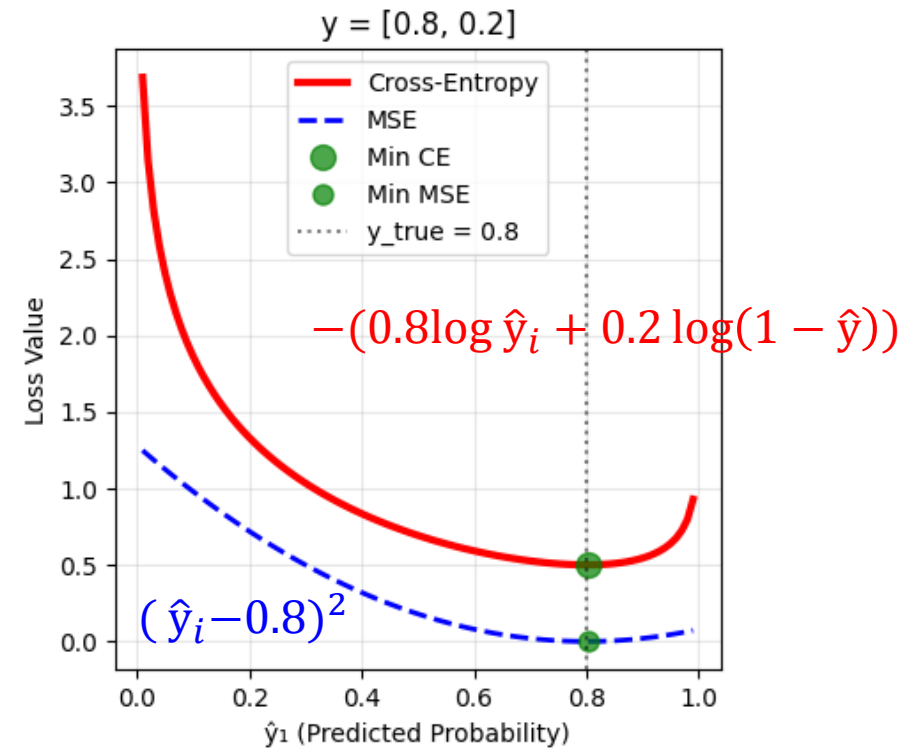
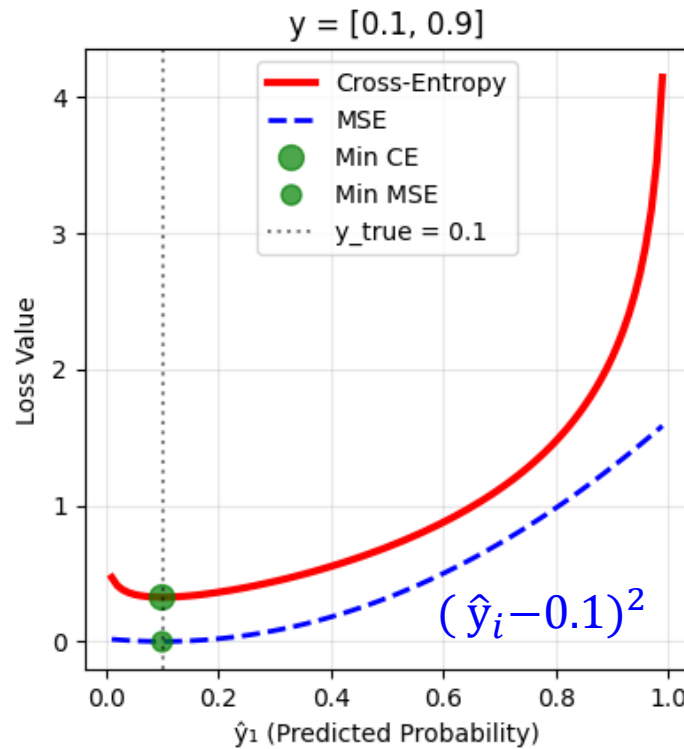
❖ Loss Function: Cross Entropy Loss

$$\text{Cross Entropy Loss} = H(y, \hat{y}) = - \sum_{i=1}^C y_i \log \hat{y}_i$$

$$-(0.5 \log \hat{y}_i + 0.5 \log(1 - \hat{y}))$$



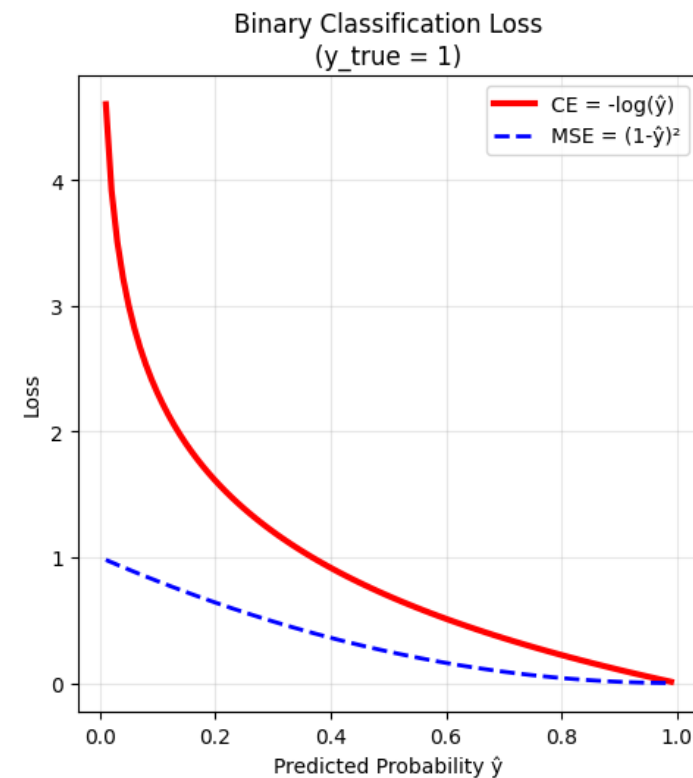
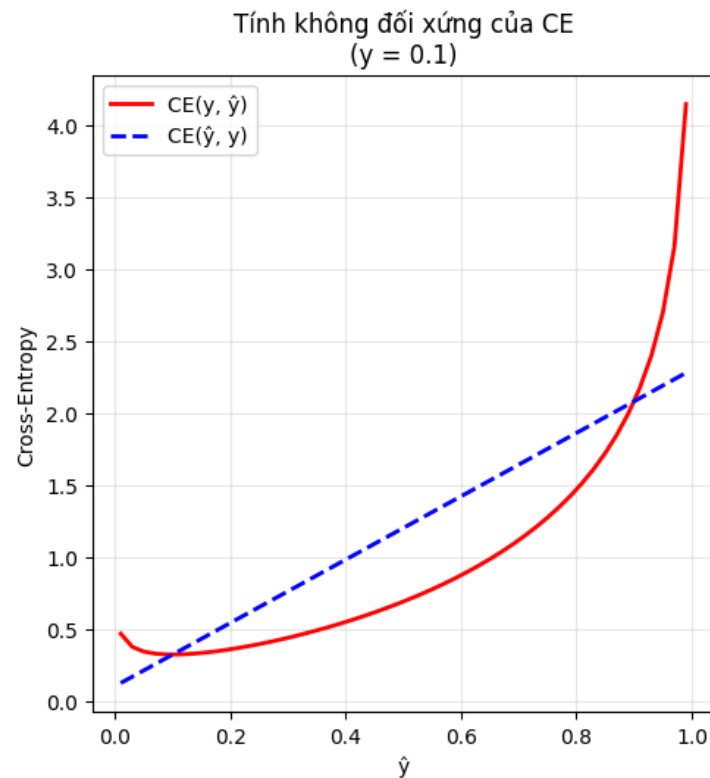
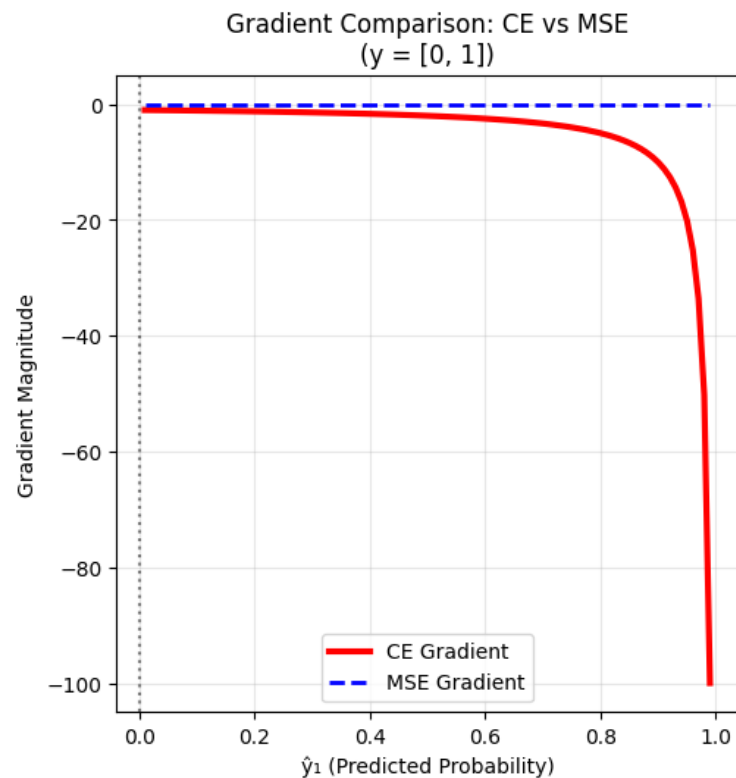
$$-(0.1 \log \hat{y}_i + 0.9 \log(1 - \hat{y}))$$



Softmax Regression

❖ Loss Function: Cross Entropy Loss

$$\text{Cross Entropy Loss} = H(y, \hat{y}) = - \sum_{i=1}^C y_i \log \hat{y}_i$$



Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is
Hadamard
division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$\mathbf{y} = [0]$$

One-hot encoding for label

$$\mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\mathbf{y} = 1 \rightarrow \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} b_0 & b_1 \\ w_0 & w_1 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

$$\eta = 0.1$$

Hours	Pass
0.5	0
1.0	0
1.5	1
2.0	1



Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is
Hadamard
division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is Hadamard division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

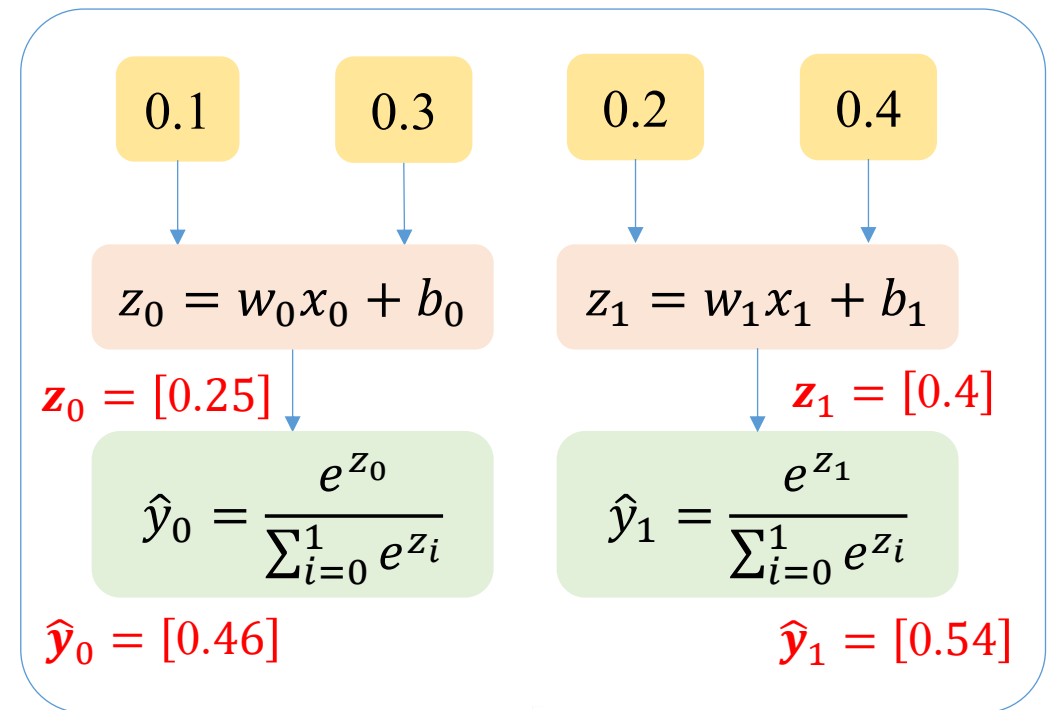
$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is Hadamard division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

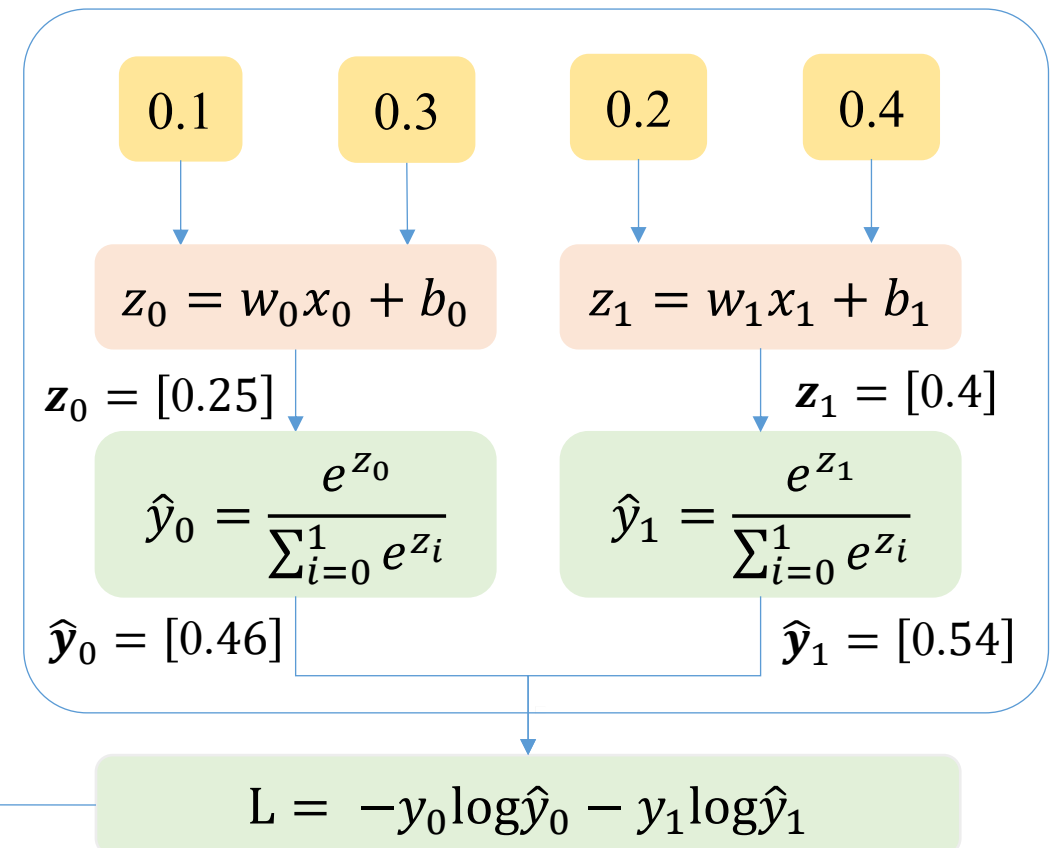
$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



$$L = [0.77]$$

Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}} \quad \begin{array}{l} \emptyset \text{ is} \\ \text{Hadamard} \\ \text{division} \end{array}$$

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

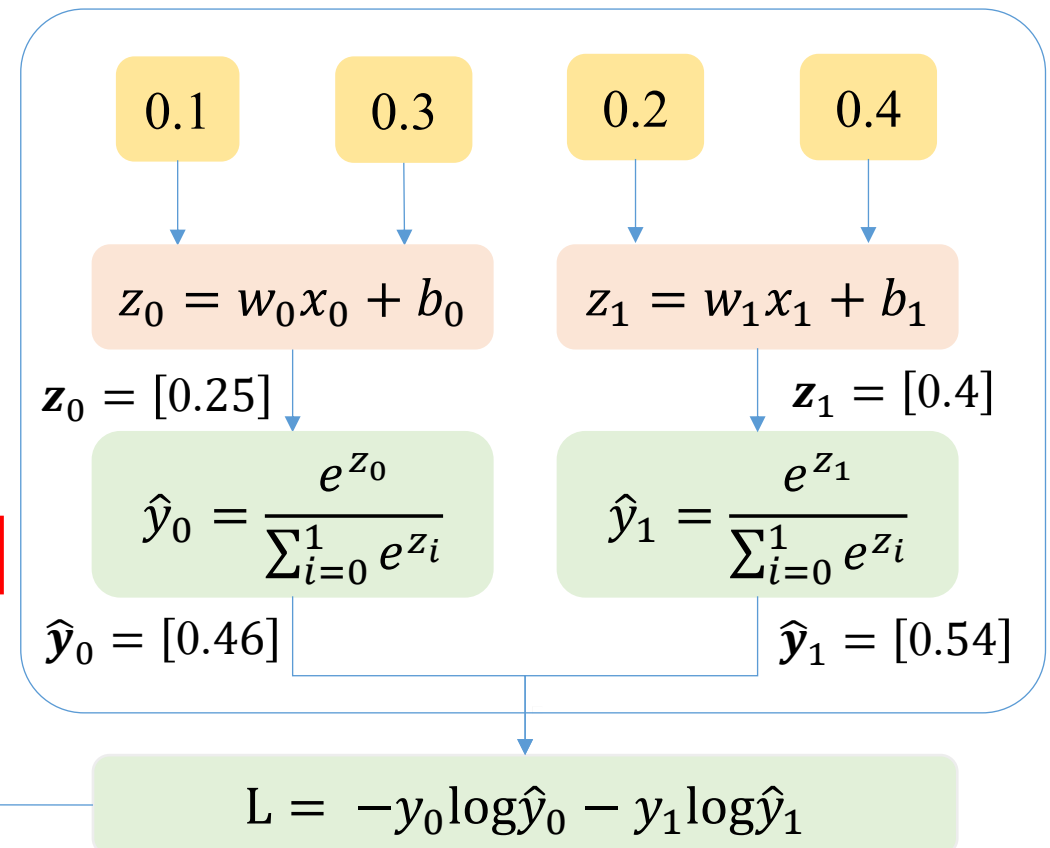
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} L &= \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T \\ &= \begin{bmatrix} -0.54 & 0.54 \\ -0.27 & 0.27 \end{bmatrix} \end{aligned}$$

$$L = [0.77]$$



Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is Hadamard division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

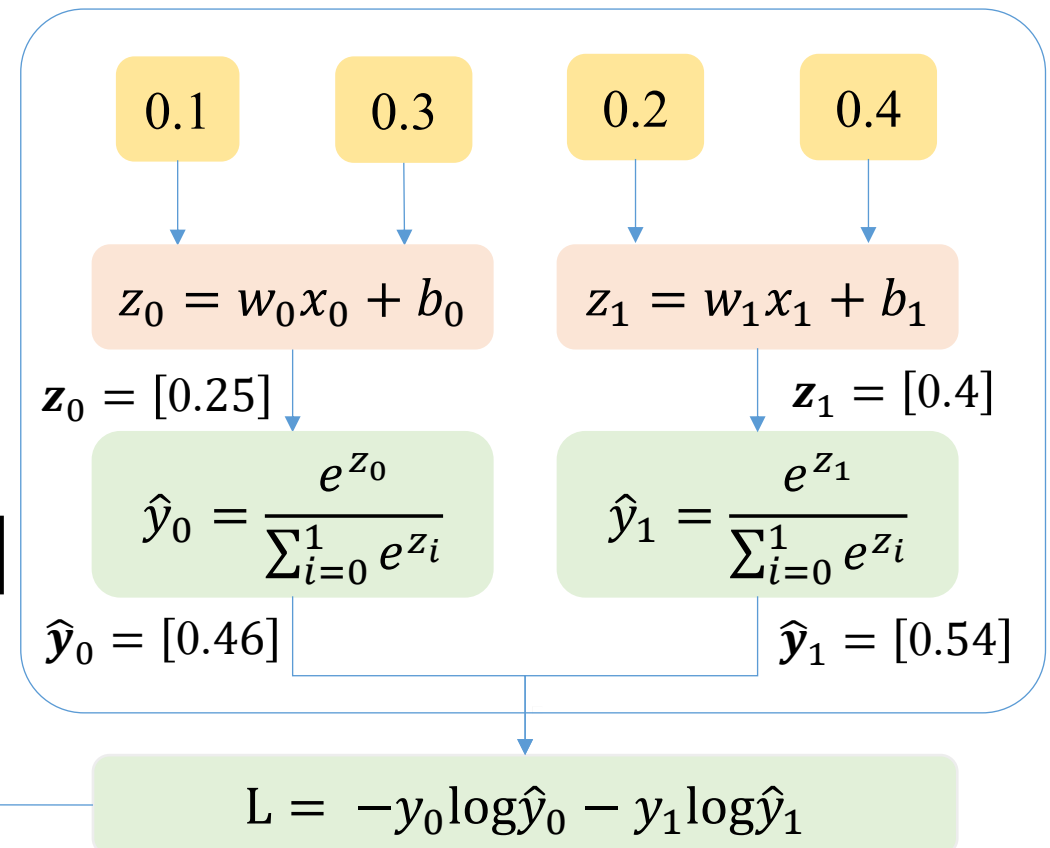
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

$$= \begin{bmatrix} 0.105 & 0.195 \\ 0.303 & 0.397 \end{bmatrix}$$

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

$$= \begin{bmatrix} -0.54 & 0.54 \\ -0.27 & 0.27 \end{bmatrix}$$

$$L = [0.77]$$



Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is
Hadamard
division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\mathbf{x} = \begin{bmatrix} 1.0 \\ 1.5 \end{bmatrix}$$

$$\mathbf{y} = [1]$$

One-hot encoding for label

$$\mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\mathbf{y} = 1 \rightarrow \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} b_0 & b_1 \\ w_0 & w_1 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

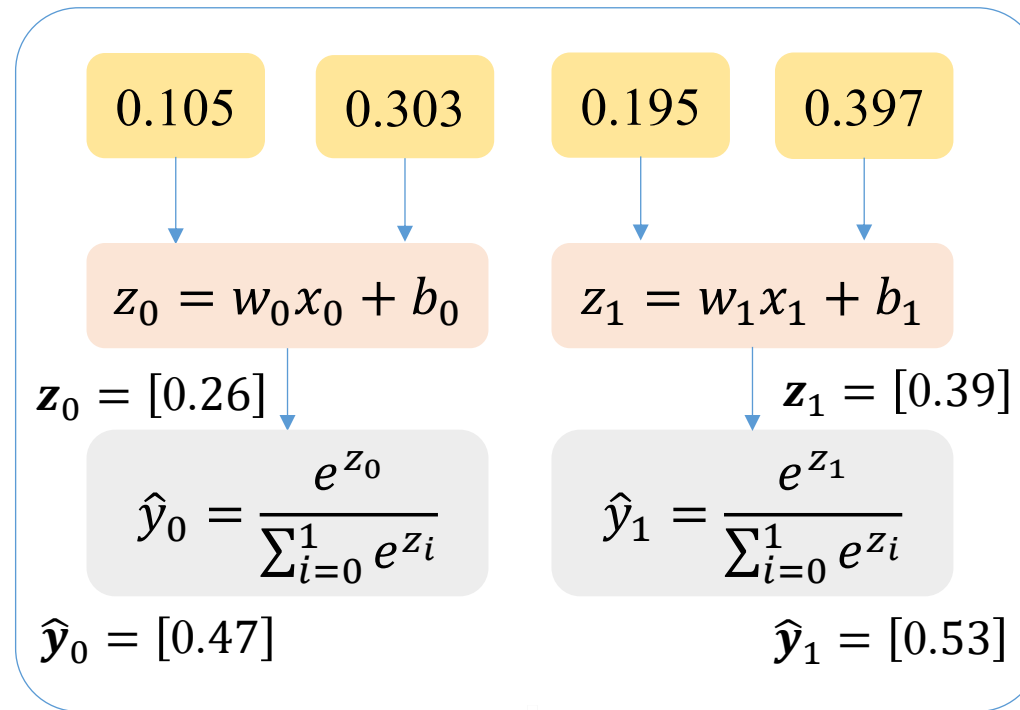
$$\eta = 0.1$$

Hours	Pass
0.5	0
1.0	0
1.5	1
2.0	1

Softmax Regression

❖ Softmax Regression using Gradient Descent

Hours	Pass
0.5	?



$$y_{pred} = 1$$

Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$z = \theta^T x$$

$$d = [1 \dots 1] e^z$$

$\emptyset$ is
Hadamard
division

$$\hat{y} = e^z \emptyset d$$

3) Compute loss (cross-entropy)

$$L(\theta) = -y^T \log \hat{y}$$

4) Compute derivative

$$\nabla_{\theta} L = x(\hat{y} - y)^T$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate

```

▶ X = np.array([
    [0.5],
    [1.0],
    [1.5],
    [2.0],
    [2.5],
    [3.0],
    [3.5],
    [4.0],
])
Y = np.array([0, 0, 1, 0, 0, 1, 1, 1])

```

```

▶ def convert_one_hot(y, k):
    one_hot = np.zeros((len(y), k))
    one_hot[np.arange(len(y)), y] = 1
    return one_hot

n_classes = 2
Y_onehot = convert_one_hot(Y, n_classes)

```

```

▶ X = np.hstack([np.ones((X.shape[0], 1)), X])

```


Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is
Hadamard
division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

```
▶ x = X[0].reshape((2,1))
  y = Y_onehot[0].reshape((2,1))
  x, y
```

```
... (array([[1. ],
           [0.5]]),
      array([[1.],
           [0.])))
```

```
▶ theta = np.array([[0.1, 0.2], [0.3, 0.4]])
  theta
```

```
... array([[0.1, 0.2],
           [0.3, 0.4]])
```

Softmax Regression

❖ Softmax Regresison using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}} \quad \begin{array}{l} \emptyset \text{ is} \\ \text{Hadamard} \end{array}$$

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d} \quad \begin{array}{l} \text{division} \end{array}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

```
# define softmax function
def softmax_function(z):
    return np.exp(z) / np.sum(np.exp(z))
```

```
# compute y_hat
def predict(x, theta):
    z = theta.T.dot(x)
    y_hat = softmax_function(z)
    return z, y_hat
```

```
z, y_hat = predict(x, theta)
z, y_hat
```

```
... (array([[0.25],
           [0.4 ]]),
      array([[0.46257015],
           [0.53742985]]))
```

Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}} \quad \begin{array}{l} \emptyset \text{ is} \\ \text{Hadamard} \end{array}$$

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d} \quad \begin{array}{l} \text{division} \end{array}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



compute loss

```
def compute_loss(y_hat, y):
    loss = -np.log(y.T.dot(y_hat))
    return loss
```

```
loss = compute_loss(y_hat, y)
loss
```

```
... array([[0.77095705]])
```

Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is Hadamard division

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



compute gradient

```
def compute_gradient(y_hat, y, x):
    gradient = x.dot((y_hat - y).T)
    return gradient
```

```
gradient = compute_gradient(y_hat, y, x)
gradient
```

```
... array([[ -0.53742985,  0.53742985],
          [-0.26871492,  0.26871492]])
```

Softmax Regression

❖ Softmax Regression using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\mathbf{z} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{d} = [1 \dots 1] \mathbf{e}^{\mathbf{z}}$$

$\emptyset$ is Hadamard

$$\hat{\mathbf{y}} = \mathbf{e}^{\mathbf{z}} \emptyset \mathbf{d}$$

division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{x}(\hat{\mathbf{y}} - \mathbf{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

$$\eta = 0.1 \quad \mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad \mathbf{y} = 0 \rightarrow \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



update weights

learning_rate = 0.01

```
def update_weight(theta, gradient, learning_rate):
```

```
    theta -= (learning_rate * gradient)
```

```
    return theta
```

```
theta = update_weight(theta, gradient, learning_rate)
```

```
theta
```

```
... array([[0.1053743 , 0.1946257 ],
          [0.30268715, 0.39731285]])
```



Softmax Regression Extend



Softmax Regression

❖ Stable Softmax Regression

What if z_i is too big?

$$P_i = f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



Softmax Regression

❖ Stable Softmax Regression

What if z_i is too big?

$$P_i = f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

```
z: [[3.e+11]
     [4.e+11]]
exp(z): [[inf]
         [inf]]
/tmp/ipython-input-2846701680.py:4: RuntimeWarning: overflow encountered in exp
  print(f"exp(z): {np.exp(z)}")
/tmp/ipython-input-2846701680.py:5: RuntimeWarning: overflow encountered in exp
  return np.exp(z) / np.sum(np.exp(z))
/tmp/ipython-input-2846701680.py:5: RuntimeWarning: invalid value encountered in divide
  return np.exp(z) / np.sum(np.exp(z))
```

e^{z_i} will be large?

Overflow

Softmax Regression

❖ Stable Softmax Regression

$$\begin{aligned}
 P_i &= f(z_i) \\
 &= \frac{e^{z_i}}{\sum_j e^{z_j}} \\
 &= \frac{e^{-c} \cdot e^{z_i}}{e^{-c} \cdot \sum_j e^{z_j}} \\
 &= \frac{e^{z_i - c}}{\sum_j e^{z_j - c}}
 \end{aligned}$$

```

▶ # define softmax function
def softmax_stable_function(z):
    c = np.max(z, axis=0, keepdims=True)
    return np.exp(z - c) / np.sum(np.exp(z - c), axis=0)

▶ # compute y_hat (stable softmax)
def predict(x, theta):
    z = theta.T.dot(x)
    y_hat = softmax_stable_function(z)
    return z, y_hat

z, y_hat = predict(x, theta)
print(f"z:\n{z}")
print(f"y_hat:\n{y_hat}")

```

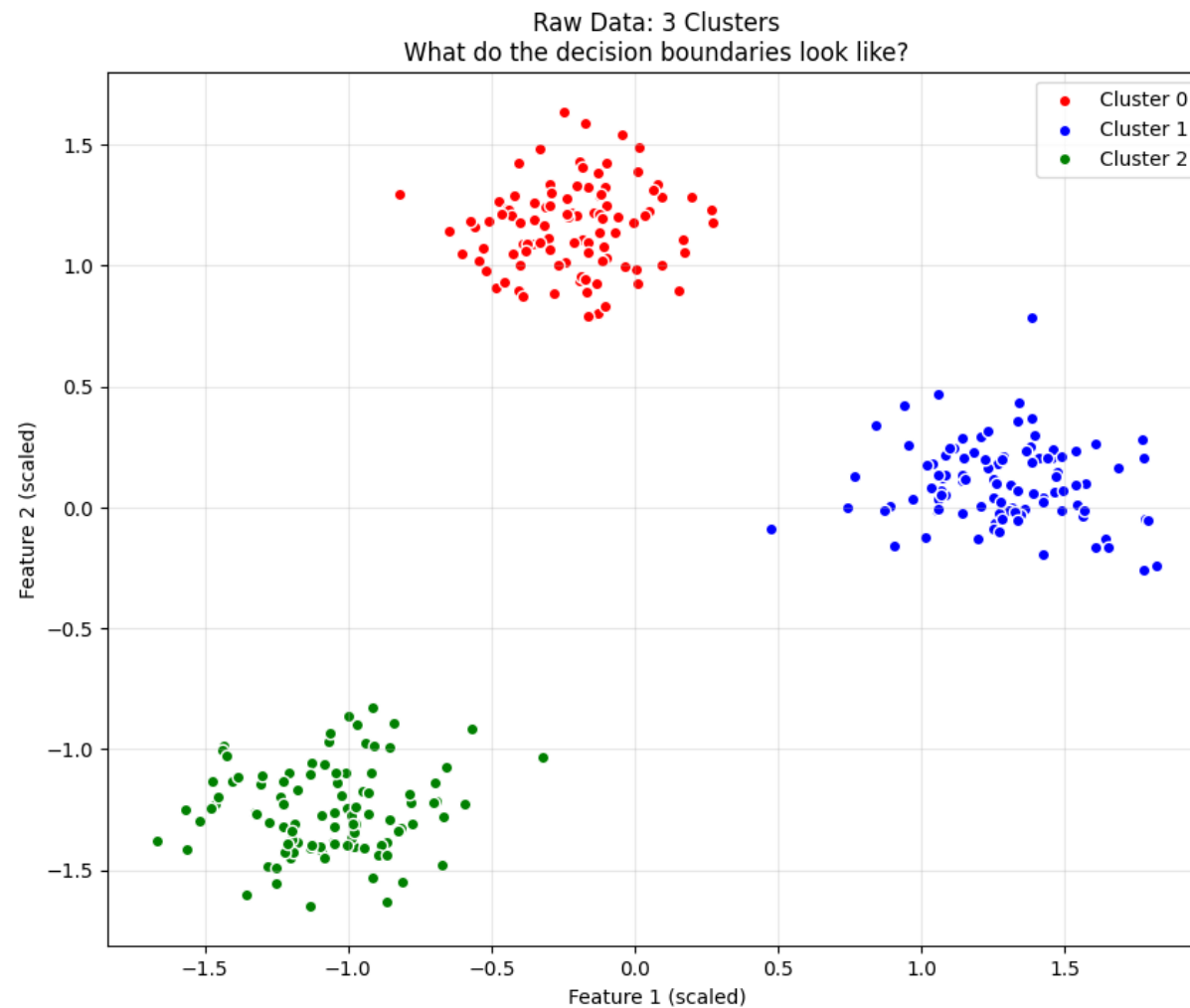
```

z:
[[3.e+11]
 [4.e+11]]
y_hat:
[[0.]
 [1.]]

```

Softmax Regression

❖ Decision Boundary is?



Softmax Regression

❖ Decision Boundary is?

$$P_i = f(z_i)$$

$$P_j = f(z_j)$$

$$P_i = P_j \Leftrightarrow f(z_i) = f(z_j)$$

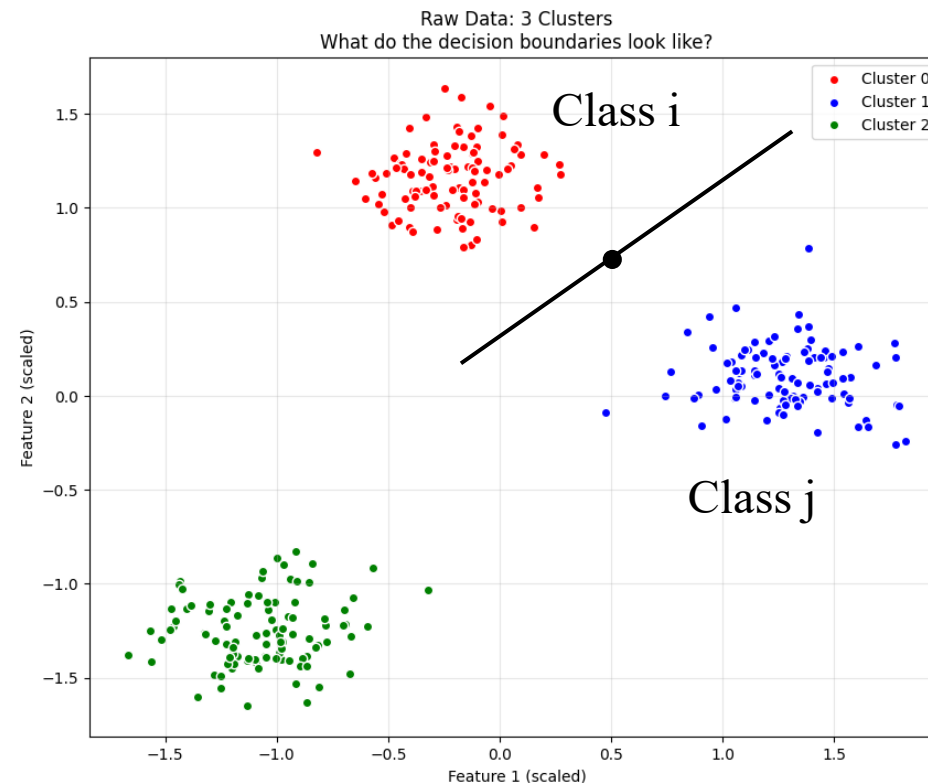
$$\Leftrightarrow \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_j}}{\sum_j e^{z_j}}$$

$$\Leftrightarrow e^{z_i} = e^{z_j}$$

$$\Leftrightarrow z_i = z_j$$

$$\Leftrightarrow \theta_i^T x = \theta_j^T x$$

$$\Leftrightarrow (\theta_i - \theta_j)^T x = 0$$



The set of points lie on
decision boundary is
linear

Softmax Regression

❖ Decision Boundary is?

$$P_i = f(z_i)$$

$$P_j = f(z_j)$$

$$P_i = P_j \Leftrightarrow f(z_i) = f(z_j)$$

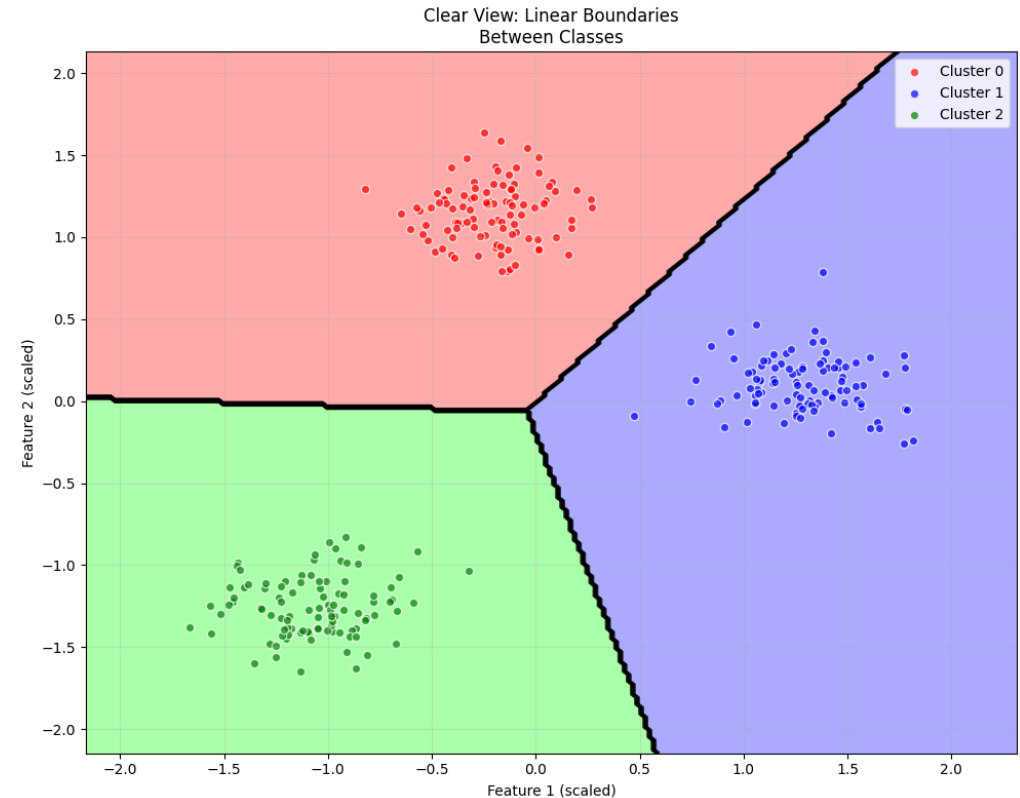
$$\Leftrightarrow \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_j}}{\sum_j e^{z_j}}$$

$$\Leftrightarrow e^{z_i} = e^{z_j}$$

$$\Leftrightarrow z_i = z_j$$

$$\Leftrightarrow \theta_i^T x = \theta_j^T x$$

$$\Leftrightarrow (\theta_i - \theta_j)^T x = 0$$



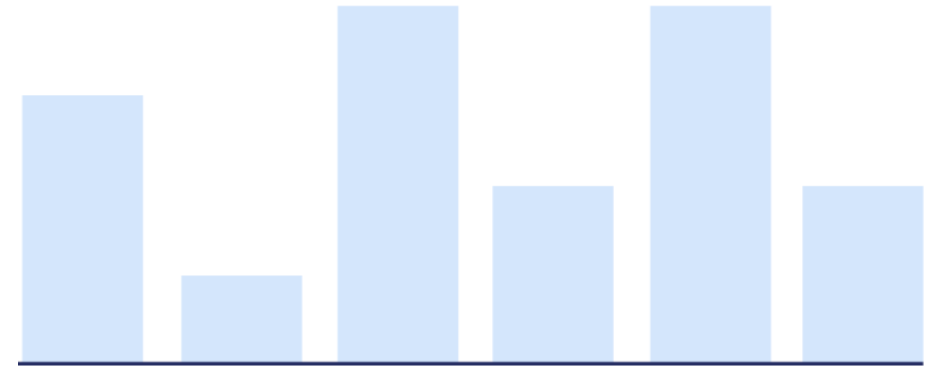
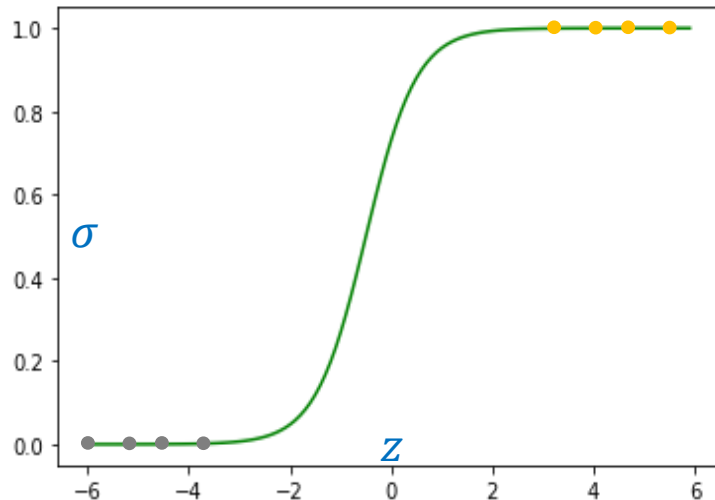
The set of points lies on
decision boundary is
linear!

Review

Logistic Regression (Review)

1. Logistic Regression
2. Sigmoid Function
3. Gradient Descent

Section 1



Softmax Regression

1. Softmax Regression
2. Softmax Function
3. One Sample
4. N Sample
5. Practice Code (numpy)

Section 2

[Quiz&Feedbacks](#)

